

A TUTORIAL FOR NEW ENGINEERS

# How to Think About Systems

We start with one engineer's Tuesday, and we finish by designing Instagram.





# Where this goes.

---

- 01 A Tuesday — one engineer, wake to sleep
- 02 The words — naming what you just watched
- 03 Protagonist and antagonist — where a design starts
- 04 Solution types — which answer is wanted
- 05 Six kits — data, compute, network, scale, reliability, security
- 06 Instagram — the graph, the design, and the worksheet you keep



PART ZERO

Before anything is a  
system, it is a Tuesday.



THE RULES OF THIS PART

# Maya has been an engineer for seven months. Today she wants one thing.

---

She wants pull request 482 merged before the release window closes at four o'clock.

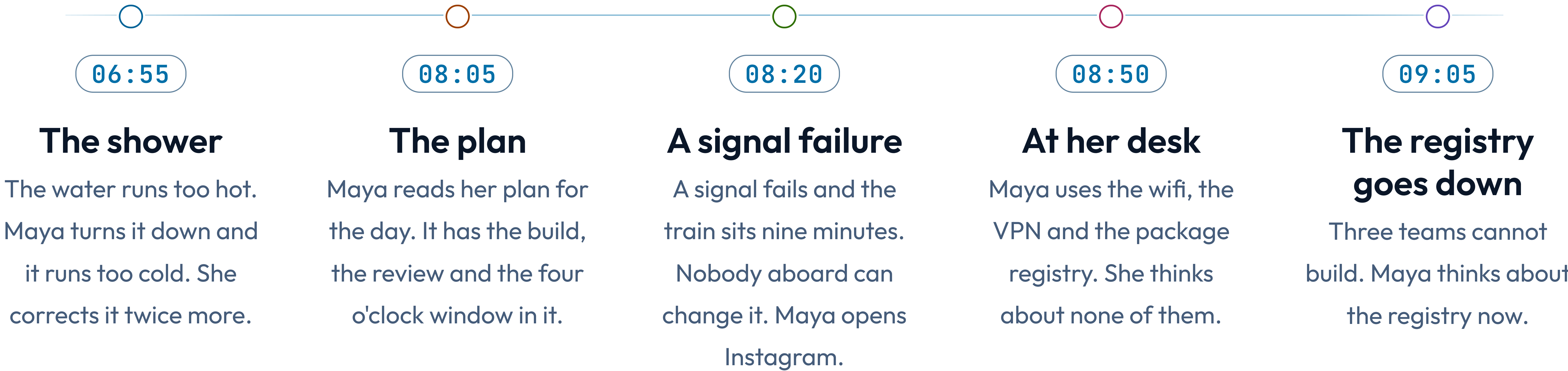
Watch her Tuesday. This part uses no systems-design words on purpose.

Everything Part one names happens here first, so you see each idea before you have a word for it.



MORNING

# Five things happen to Maya before she reaches her desk.





09:15 • STANDUP

# Four people met twice this week, and only one meeting worked.

---

On Monday each person reported to the manager in turn. That meeting ran twenty minutes and settled nothing. Today they talked to each other instead, and Maya learned in one sentence that Priya had already read the code she was about to start.

The four people did not change. Only who talked to whom changed.



*Get 482 merged before the window closes.*

MAYA WROTE THIS ON A STICKY NOTE AT 09:30. NOTHING ELSE IS WRITTEN ON IT.



09:45 · IN THE WAY

# Four limits shape Maya's day, and she chose none of them.

---

## The build takes twelve minutes

Every push costs twelve minutes. Maya cannot shorten that or skip it.

## The team is three people

Nobody on the team is free to review her code this morning.

## The one reviewer is asleep

He is six time zones away and will not read anything before three o'clock.

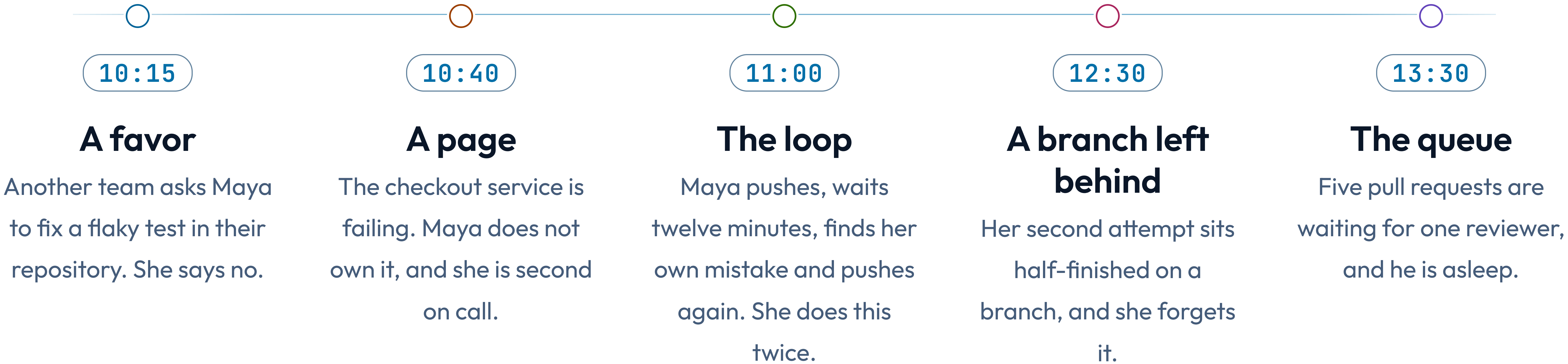
## Production data stays in production

She may not copy it to her laptop, even to reproduce the bug.



MIDDAY

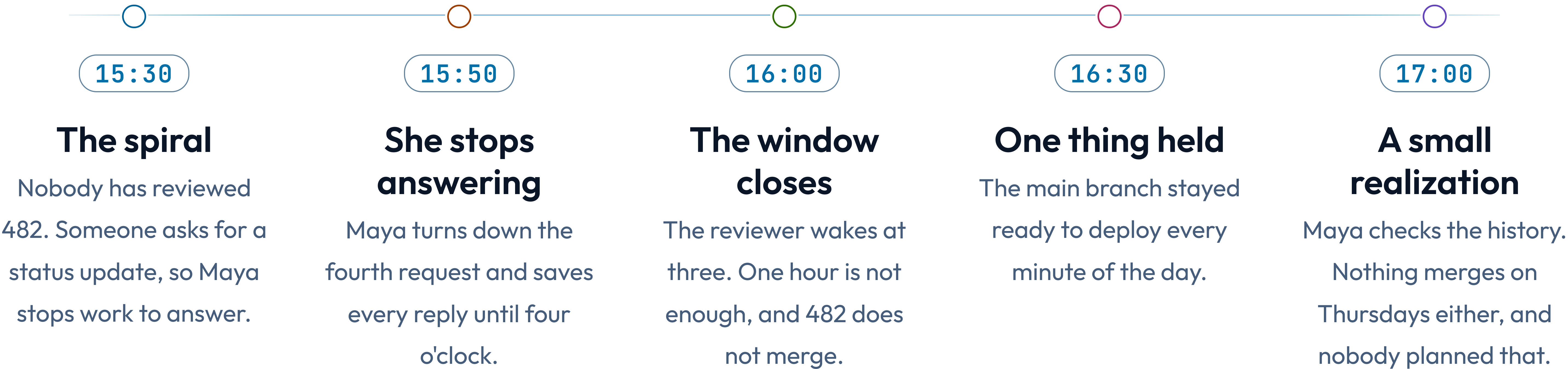
# By lunchtime, other people decide what Maya works on.





AFTERNOON

# Maya spends the afternoon answering questions about the work she is not doing.





22:40 • THE REVEAL

# You just watched a system run for sixteen hours.

---

It had a goal it did not meet, and one hour that decided whether it would. Two things fed themselves in circles, one of them in her shower. Something invisible held the whole thing up until nine o'clock, when it stopped. One promise never broke, on the day everything else did.

The reviewer woke at three and the window shut at four. Everything Maya did that day reached the outcome only through that hour. She spent twenty of its sixty minutes answering questions about 482, instead of waiting ready to act on whatever came back.

You have watched all ten. You do not yet have the words.



# 01

---

PART ONE

Now we give each of  
those things its name.



WORD ONE • SYSTEM

# A system is parts, connected, doing something together.

Maya's morning is one. The shower, the train, the laptop and the registry between them do what none of them does alone: put Maya at a desk, able to work, at nine.

IN MAYA'S DAY

Nothing on that list is a system. The order she does them in is.

LISTING THE PARTS IS EASY

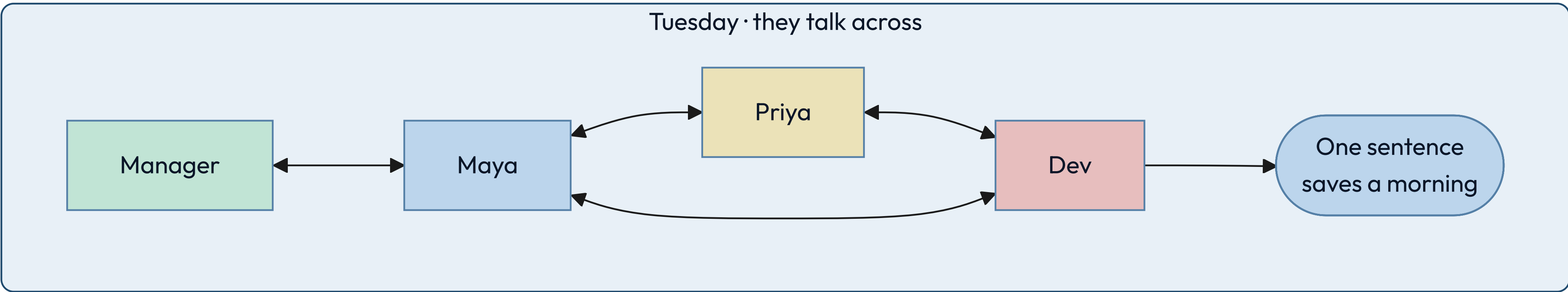
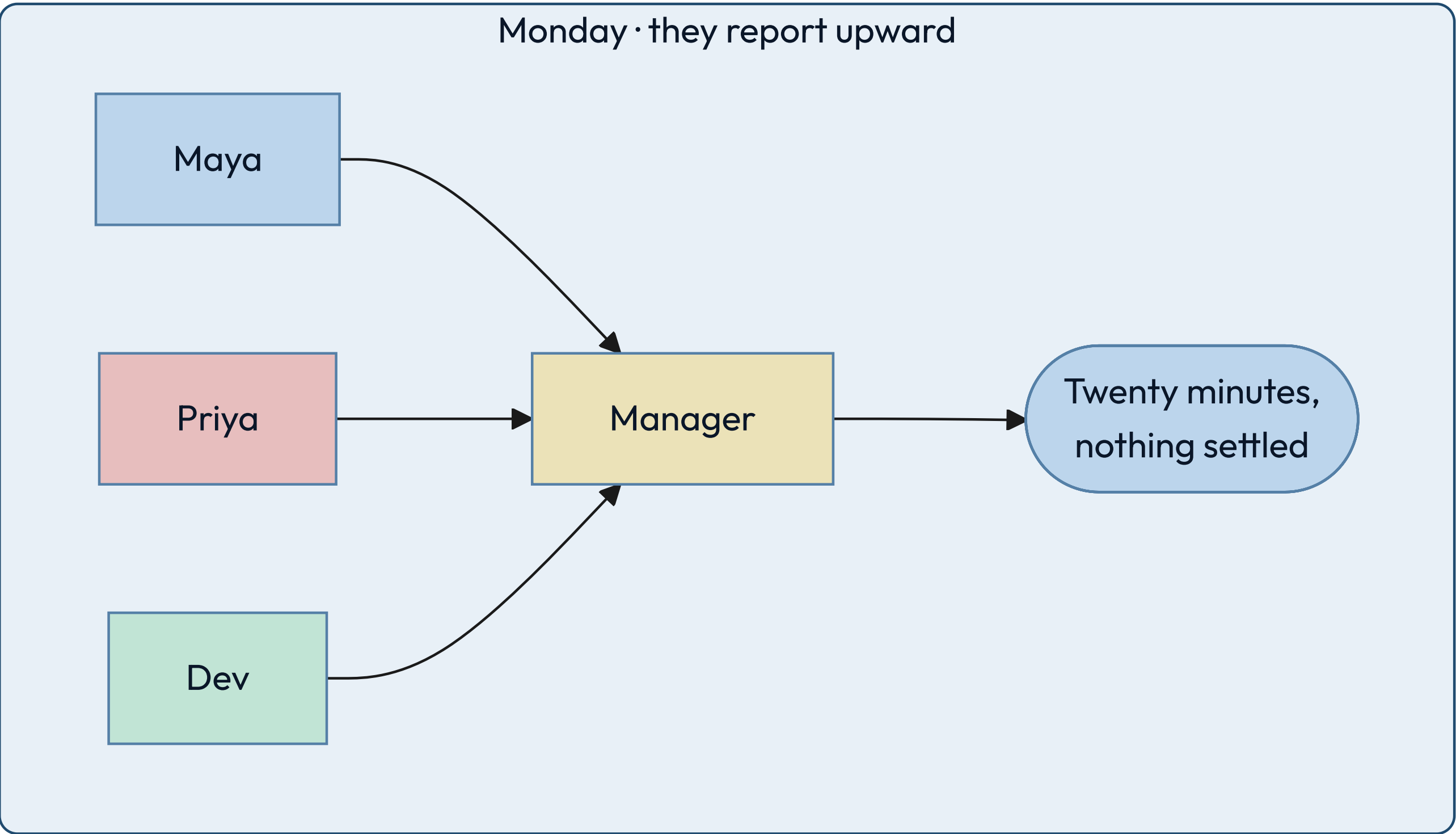
Shower, train, laptop, registry. Anyone can write that list.

THE CONNECTIONS ARE THE SYSTEM

Change what depends on what, and the behavior changes with the same parts.



# Rewire a system without changing a single part, and it behaves differently.





WORD TWO • PURPOSE

## A purpose becomes visible at the moment you miss it.

Maya wrote it on a sticky note at nine thirty: get 482 merged before four. At four o'clock it had not merged, and the gap between those two facts is the clearest thing in her whole day.

### IN MAYA'S DAY

She wrote hers down at 09:30, which is why she could tell at 16:00 that she had missed it.

### WRITE IT DOWN OR YOU WILL DRIFT

An unwritten purpose gets quietly replaced by whatever arrived most recently.

### A SYSTEM'S PURPOSE IS WHAT IT DOES

Not what its owners say it does. Watch the outputs, not the mission statement.



WORD THREE • BOUNDARY

# The boundary is the line between what you change and what you ask for.

At ten fifteen another team asked Maya to fix a flaky test in their repository. She said no. That refusal is the boundary, and she could feel exactly where it was because saying no was uncomfortable.

## IN MAYA'S DAY

She could decline the favor. She could not decline the release window.

## INSIDE IS WHAT YOU CHANGE

Your code, your schema, your deploys, the alerts that wake you.

## OUTSIDE IS WHAT YOU NEGOTIATE

The reviewer's time zone, the registry, the build, another team's repository.



WORD FOUR • ENVIRONMENT

# The environment is everything that arrives without asking.

A signal failure held her train for nine minutes. A page pulled her into an outage in a service she does not own. Neither was load, neither was a bug, and neither was hers.

## IN MAYA'S DAY

She had no say over either. She did have a say over what happened to 482 while she was gone, and had not decided it in advance.

## IT IS NOT THE SAME AS LOAD

Regulation, a partner's outage and a colleague's time off are environment too.

## YOU DESIGN FOR IT, NOT AGAINST IT

You cannot stop the page. You can decide what happens to 482 when it comes.



WORD FIVE • PROCESS

# A process is a repeatable transformation, and it runs at a rate.

Push, wait twelve minutes for the build, read a comment, fix, push again. Maya ran that loop twice. Every process has inputs, a transformation, outputs, a rate, and something it leaves behind.

## IN MAYA'S DAY

Twenty-four minutes of her day were spent waiting for a machine, and she chose none of them.

## THE RATE IS PART OF THE DEFINITION

"Run the tests" is not a process until you say twelve minutes, and how often.

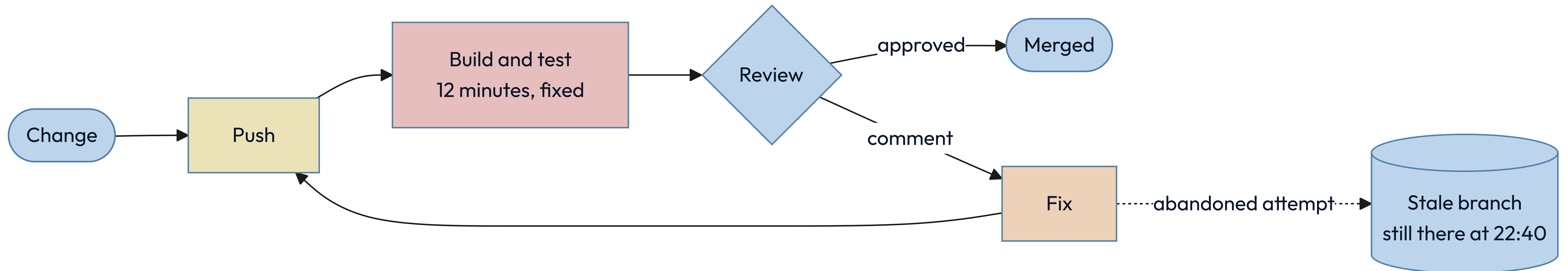
## LEFTOVER STATE IS WHERE BUGS LIVE

Her abandoned branch is the same shape as a half-applied database migration.



11:00 • THE LOOP, DRAWN

# Every process leaves something behind, and that is the part nobody draws.





WORD SIX • MODEL

**A model leaves things out on purpose. What beats you is the thing you left out by accident.**

Maya's plan for Tuesday was a model. It held the build time, the review, and the four o'clock window. It did not hold the fact that her reviewer sleeps six time zones away, and that one gap decided the day.

IN MAYA'S DAY

Her plan was right about everything in it and silent about the thing that beat her.

EVERY DIAGRAM IN THIS DECK IS  
A MODEL

Each one throws away something true so you can see one thing clearly.

SAY WHAT YOU LEFT OUT

An omission you can name is a simplification. One you cannot name is a bug.



WORD SEVEN • CONSTRAINT

# A constraint is a limit that removes options rather than adding caveats.

Four limits shaped Maya's day, and each came from somewhere different. A twelve-minute build. A team of three. A reviewer asleep six time zones away. A rule about production data.

## IN MAYA'S DAY

Physical, economic, human and legal limits, all four of them landing before ten in the morning.

## REAL CONSTRAINTS CARRY NUMBERS

"The build is slow" is a complaint.  
"Twelve minutes" is something you can design against.

## SOME YOU CHOSE, AND CAN REVISIT

A team of three is a decision. The speed of the build is arithmetic.



WORD EIGHT • INVARIANT

**An invariant is a claim that stays true on the day everything else fails.**

Main was deployable every minute of Maya's Tuesday. It was true while the registry was down, while she was paged, and at four o'clock when 482 did not land.

IN MAYA'S DAY

She missed her goal and broke no invariant, and those are two different kinds of bad day.

WRITE THEM AS SENTENCES

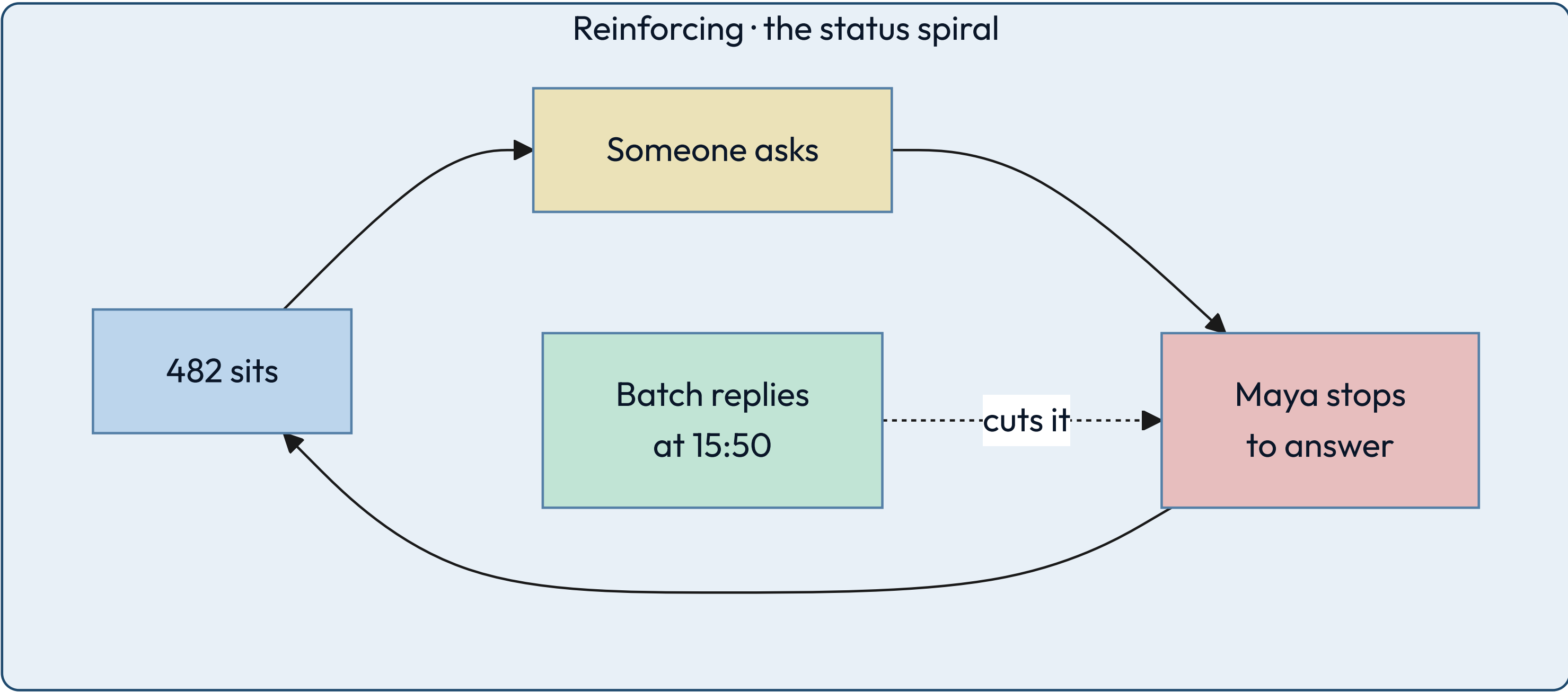
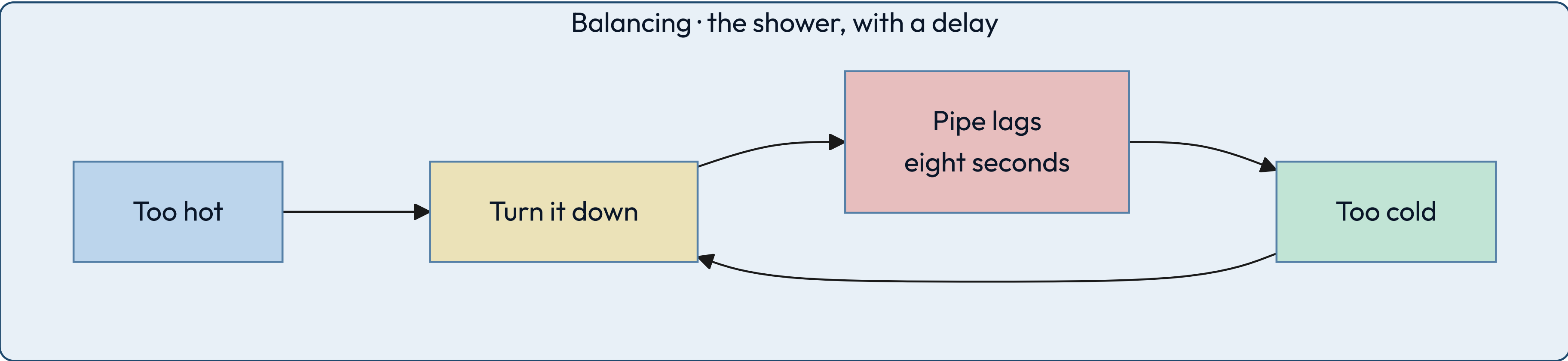
"Main is always deployable." "A balance is never negative." Then name the alert.

THEY CONSTRAIN OTHER PEOPLE

A real invariant changes what your teammates are allowed to merge.



# Two loops look identical until you find the sign.



✦ A balancing loop pushes back toward a target, and the eight-second delay in the pipe is exactly why she overshoots. A reinforcing loop pushes harder in the direction it is already going.



# Infrastructure is what you only notice on the day it stops.

Wifi, the VPN, the package registry, the build fleet, the identity provider. Maya used every one of them before nine o'clock and thought about none of them until 09:05, when the registry went down and three teams stopped.

## THE TEST

*If it vanishing stops several unrelated things at once, it is infrastructure.*

## BORING IS THE REQUIREMENT

It earns its place by being predictable, never by being interesting.

## IT IS SOMEONE ELSE'S SYSTEM

Your infrastructure is another team's product, with its own boundary and invariants.



17:00 • THE LAST WORD

# Emergence is a pattern the parts fall into, that none of them contains.

---

Nothing merges on Thursdays. Maya checked six weeks. It is a rhythm, and nobody built a rhythm.

Four things feed each other rather than add up. Work that misses the window waits at the front of tomorrow. A review that comes back with comments sends the same change to the back of the queue. The queue is served one hour a day. And nobody ships into a weekend, so Thursday is the last window of the week — the day the backlog is deepest and the day it has to clear.

No policy names Thursday. You find it by watching the queue for six weeks.



# Five words describe the thing itself.

You did not learn these from a definition. You watched each one happen first, which is the order that sticks.

|    |             |                                   |                          |
|----|-------------|-----------------------------------|--------------------------|
| 01 | System      | Parts, connected, with a purpose. | The whole morning.       |
| 02 | Purpose     | What it does, not what it says.   | The gap at four o'clock. |
| 03 | Boundary    | What you change, not ask for.     | The favor she declined.  |
| 04 | Environment | What arrives uninvited.           | The page.                |
| 05 | Process     | A transformation with a rate.     | Push, build, review.     |



# Three more are the tools you think with.

You never handle the system itself. You handle a drawing of it, its limits and its promises — standing on infrastructure you notice only when it stops.

|    |            |                               |                       |
|----|------------|-------------------------------|-----------------------|
| 01 | Model      | A simplification you chose.   | Her plan for the day. |
| 02 | Constraint | A limit that removes options. | Twelve minutes.       |
| 03 | Invariant  | What must never go false.     | Main is deployable.   |



THE TRANSLATION

# Every move in Maya's day has a name in software.

| IN HER TUESDAY                   | IN A SYSTEM              | WHAT IT DECIDES                            |
|----------------------------------|--------------------------|--------------------------------------------|
| Five pull requests, one reviewer | The bottleneck           | Where any improvement has to land          |
| Declining the fourth status ask  | Admission control        | Whether load sheds or the system collapses |
| A twelve-minute build            | A fixed cost per attempt | How many attempts a day can hold           |
| The abandoned branch             | Leftover state           | What a retry finds when it arrives         |
| Nothing merges on Thursdays      | Emergence                | What no single owner can fix               |



# 02

---

PART TWO

Every design starts with  
someone who wants something,  
and something in the way.



THE FRAME

# Name the person and the force, and everything downstream has an answer.

---

Juniors skip both, almost every time. They write "users" instead of one person with one goal, and they write "scale" instead of a force with a number on it. Neither produces a single design decision.

The protagonist is who the system is for. The antagonist is what makes serving them hard. Everything downstream — the purpose, the boundary, the invariants, the first move — falls out of that pair.



## THE DRILL

**Two sentences, ninety seconds, and four things you no longer have to guess.**

Protagonist: <name> wants to <do X> so that <Y>.

Antagonist: But  $\langle W \rangle$  – and  $W$  is  $\langle a \text{ number} \rangle$ .

Purpose      one sentence: what counts as this working

Boundary      what is mine when W happens, and what I only ask for

Invariant      what must stay true or <name> stops trusting this

First move      what W does to the cheapest design that could work

✦ *Run it on a turnstile, then a cash machine, then the thing on your screen right now. It gets fast.*



THE DRILL, ON TUESDAY

# Maya is the protagonist of her own day, and the interruptions are the antagonist.

MAYA WANTS 482 MERGED BEFORE FOUR O'CLOCK

So that a bug affecting real users stops affecting them, and so she is not carrying it into next week.



BUT SHE IS INTERRUPTED SIX TIMES

A page, a favor, four status requests. Not one of them unreasonable, and each costs the reload, not just the minute.



## THE TRAP

# Most systems have several protagonists, and you must say which one loses.

---

Instagram has at least four: the reader opening the app, the ordinary poster, the celebrity with five hundred million followers, and the engineer carrying the pager. They want incompatible things.

Naming one as the protagonist is not a slogan, it is a decision about who waits. Say who you are not designing for, out loud, and most of the arguments later in the design turn out to be about that.



THE JOIN

The antagonist chooses which kind of solution you are allowed to build.

| THE ANTAGONIST IS...                | YOU ARE BUILDING           | BECAUSE                         |
|-------------------------------------|----------------------------|---------------------------------|
| Nobody knows if they exist          | An MVP                     | The risk is demand, not load    |
| Growth — the protagonist multiplies | A scaled system            | The limit is real and namable   |
| Cost on a path you have profiled    | An optimized system        | The measurement came first      |
| Physics — you are near a real bound | An optimal system          | Only here is a proof worth it   |
| A person who wants in               | Security work, at any rung | It is not a rung, it is a floor |



INSTAGRAM · THE CASTING

# Our protagonist reads on a phone and gives us about a second.

She opens the app a dozen times a day, on a cellular network, usually while doing something else. She wants photographs from the people she chose, recent, ranked, and hers. The poster is a supporting character: he tolerates a spinner, and every time the design has a choice, work goes onto his side.

## THE READER'S DEMAND

A page on her phone in under a second, which is 200 ms of server time plus the network.

## THE POSTER CAN WAIT

Uploading, transcoding and delivery may all take their time. Nobody is watching.

## CASTING DECIDES THE DESIGN

It is why the feed is built when someone posts, not when someone reads.



# The antagonist is not scale. It is the shape of the follow graph.

Scale is a quantity, and quantities have a price you can pay. The thing you cannot buy your way out of is a distribution. Follower counts are heavy-tailed: almost everyone has a few hundred, and a handful have hundreds of millions. No single account looks like the average, and the far end sits six orders of magnitude from the middle.

## THE NUMBER THAT MATTERS

*The median account has about 150 followers. The largest has five hundred million.*

## ONE ALGORITHM CANNOT SERVE BOTH

That gap is why the design ends up with two paths instead of one.

## HOLD ON TO THIS

Every hard decision in Part five is this one fact again.



# 03

---

## PART THREE

"Design Instagram" is  
five different questions.



BEFORE THE BOXES

# Before you design anything, say which kind of answer is wanted.

---

The same words — design a photo sharing app — have five legitimate answers that share almost no architecture. A build that takes a week and a build that takes two years are both correct, for different questions.

Gall's law says it plainly: a complex system that works is invariably found to have evolved from a simple system that worked. You climb this ladder. You do not parachute onto it.



Each rung costs more and commits harder than the one below it.

You move up when the rung you are standing on stops paying, and you move up one rung at a time.

|    |             |                               |                           |
|----|-------------|-------------------------------|---------------------------|
| 01 | MVP         | Buys information fast.        | Does anyone want this?    |
| 02 | Scaled      | Holds up as load grows.       | Will it survive success?  |
| 03 | Optimized   | Cuts cost on a known path.    | Where is the money going? |
| 04 | Optimal     | Provably best, for one model. | What is the real limit?   |
| 05 | Specialized | An advantage nobody can copy. | What can only we do?      |



RUNG ONE • MVP

# An MVP is an experiment wearing a product's clothes.

Its job is to produce a decision, not to last. Every hour spent making it durable is an hour spent on a system you may correctly delete next month.

## THE TELL

The riskiest thing is whether anyone wants it, and being wrong is cheap.

## BUY SIMPLICITY, NOT CAPACITY

One database, one service, one region, boring technology, nothing clever anywhere.

## NAME THE EXIT IN ADVANCE

Write down the number that means "stop, this now has to be built properly."



RUNG TWO • SCALED

# A scaled system survives ten times the load without a rewrite.

You have stopped buying information and started buying headroom. The question moves from "does it work" to "what breaks first, and how do I move that limit."

## THE TELL

Growth is real, and the current design has a ceiling you can point at.

## A BIGGER BOX FIRST, THEN THE FOUR MOVES

Vertical scaling is not one of the four; it is what buys time before you need them, and it is reversible. Then reduce, duplicate, defer — and spread last, because partitioning is the one you cannot undo cheaply.

## COST PER UNIT STARTS COUNTING

Cost per request stops being noise and becomes the second constraint on every choice.



RUNG THREE • OPTIMIZED

# Optimizing means moving one measured number on one hot path.

Optimization without a profile is decoration. You need the measurement first, the target second, and a willingness to accept the complexity you are about to add.

## THE TELL

A profile shows which tenth of the work is most of the cost.

## PREMATURE IS HALF THE QUOTE

Knuth also wrote that we should not pass up the critical three percent. Both halves.

## COMPLEXITY IS THE INVOICE

Each optimization narrows the assumptions the system is allowed to break.



RUNG FOUR • OPTIMAL

# Optimal means provably best against an objective you wrote down.

Rarer than it sounds. It needs a stated objective, a stated model, and a proof or a bound — and it is optimal only for the assumptions you fixed in place.

## THE TELL

The objective is a function and the constraints are inequalities, on paper.

## THE PROOF IS AGAINST A MODEL

Change the assumptions and the optimal answer changes underneath you.

## IT AGES BADLY

An optimal design pinned to last year's hardware is a legacy system with a certificate.



# A specialized system trades generality for something nobody can copy.

Custom silicon, a purpose-built storage engine, a scheduler that knows your physics. You give up flexibility, portability and hiring pool for a capability the market cannot sell you.

## THE SIGNAL

*The advantage is durable, measurable, and central to why customers choose you.*

## THE COST NEVER ENDS

You now maintain what everyone else gets free from a vendor, forever.

## ALMOST NOBODY IS HERE

Most teams reaching for this rung needed the optimized one and got excited.



THE RULE

# Climb one rung at a time, and only on evidence.

MOVE UP WHEN THE CURRENT RUNG  
FAILS ON A MEASUREMENT

A number, a profile, a named risk.  
Not a feeling that things are  
getting big.

MOVE UP EXACTLY ONE RUNG

Jumping from MVP to optimal  
buys rigor for assumptions  
nobody has tested yet.

MOVE BACK DOWN WHEN THE EVIDENCE  
CHANGES

A rewrite that simplifies is a  
legitimate move, not an  
admission of anything.



# A design is finished when taking one more thing out would break it.

Antoine de Saint-Exupéry wrote that perfection arrives not when there is nothing left to add, but when there is nothing left to take away. Dieter Rams said it in three words: less, but better. Both are tests you can run on a whiteboard at four in the afternoon.

## RUN IT ON EVERY BOX

*Delete it on paper and follow what happens. If nothing downstream changes, it was never holding anything up.*

---

## RUN IT ON EVERY NUMBER

A figure nobody can act on is decoration. Turn it into a threshold or take it out.

---

## LESS, BUT BETTER, IS NOT LESS

Rams designed for decades of use. Removing something the system needs is not restraint, it is a defect.



# 04

---

PART FOUR

**Six kits: fifteen things  
you can reach for, and the  
invariants behind all of them.**



# Reach for it when, walk away when, and the constraint you inherit.

---

One shape for every entry you choose between — a store, a runtime, a delivery tier, a quota — so you can hold two options side by side without re-reading a manual. Scale and reliability are mostly practices rather than choices, so those two kits run on diagrams, patterns and invariants instead. Every kit opens with a diagram and closes with its invariants.

The entries name concepts, not products. Products turn over every few years. The constraint that an append-only store puts on your reads does not.

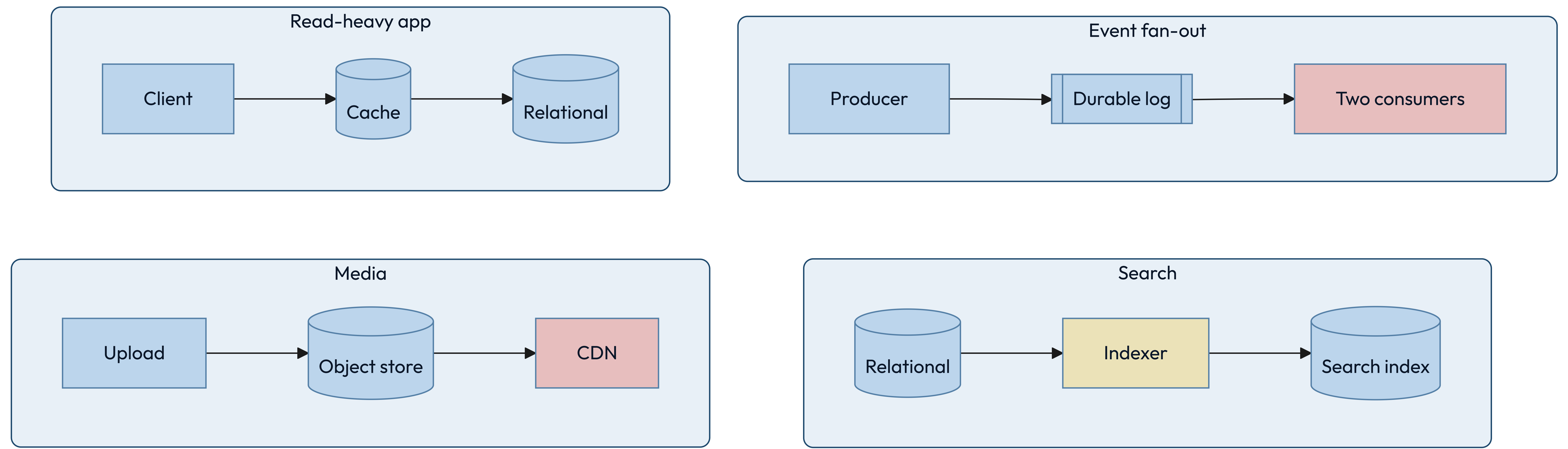


DATA

**Every storage choice is a bet  
about how you will read it later.**



# Four shapes cover almost everything you will store.





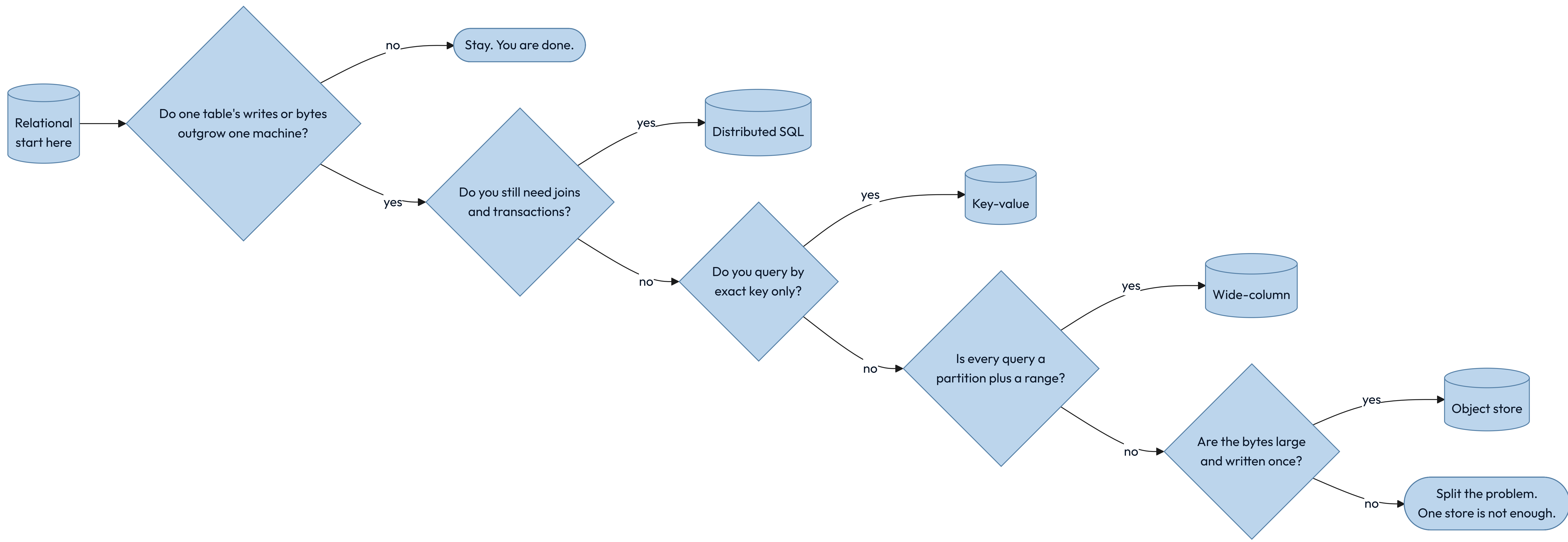
# The product name is the last thing you decide about a store.

Answer these before anyone says a brand. Most database arguments are really a disagreement about question two.

|    |             |                             |                            |
|----|-------------|-----------------------------|----------------------------|
| 01 | Shape       | How the data is structured. | Rows, documents, or edges? |
| 02 | Access      | How it is read and written. | Lookups, ranges, or scans? |
| 03 | Consistency | What a reader may see.      | Must it be the latest?     |
| 04 | Scale       | Size, throughput, growth.   | Does it fit one machine?   |
| 05 | Cost        | Money, operations, skill.   | Who runs it at 3am?        |



# Start at relational and branch out. That is how the decision actually gets made.





# A relational store is the default until you can name why it is not.

## Reach for it when

Entities relate, writes touch several at once, and the questions will keep changing.

## Walk away when

One table outgrows one machine's writes, or the schema genuinely differs per row.

## The constraint you inherit

Joins and transactions need a coordinator. Self-sharding loses both; a distributed engine charges a round trip.



# An engine that partitions itself is the first answer now, not the last resort.

## Reach for it when

One table outgrew one machine and you still want joins, transactions and a schema.

## Walk away when

Every query is a single exact key, or the budget cannot carry cross-partition coordination.

## The constraint you inherit

A transaction across partitions pays a round trip, so related rows still belong together.



# A key-value store is a hash map with an operations team.

## Reach for it when

You always know the exact key, and handing the value back is the store's only job.

## Walk away when

You need to ask any question at all about what is inside the value.

## The constraint you inherit

There is no second way in. Every new query means a new key you write and maintain yourself.



# Choosing a document store means choosing to keep one object whole.

## Reach for it when

A read fetches one self-contained object, and the shape varies between records.

## Walk away when

The same fact lives in many documents and has to stay consistent across them.

## The constraint you inherit

Denormalized copies. Every update to a shared fact becomes a fan-out — one write you now owe to many places.



# The same five questions, asked of every store you are choosing between.

| STORE           | ACCESS               | CONSISTENCY              | SCALES BY                   | WEAK AT                           |
|-----------------|----------------------|--------------------------|-----------------------------|-----------------------------------|
| Relational      | Key, range, join     | Strong on the leader     | Replicas, then partitioning | Cross-shard writes                |
| Distributed SQL | Key, range, join     | Strong across partitions | Horizontal                  | Cross-partition transactions      |
| Key-value       | Exact key            | Engine-specific          | Horizontal                  | Rich queries                      |
| Document        | Key, secondary index | Engine-specific          | Horizontal                  | Facts split across documents      |
| Wide-column     | Partition plus range | Tunable                  | Horizontal                  | New query patterns                |
| Graph           | Traversal            | Engine-specific          | Horizontal, with effort     | Cutting the graph across machines |



# A wide-column store buys enormous write throughput and freezes your queries.

## Reach for it when

Writes are relentless, and every read is a partition key plus a sorted range.

## Walk away when

Query patterns are still moving, or you need a transaction across partitions.

## The constraint you inherit

The primary key is the schema. Changing how you query means rewriting the data.



# A graph store is for questions that traverse, not questions that filter.

## Reach for it when

The query walks relationships of unknown depth: reachability, paths, recommendations.

## Walk away when

You have relationships but only ever join two hops. A relational store does that faster.

## The constraint you inherit

Traversals resist partitioning, so scaling out is genuinely harder here than anywhere else.



# Two stores you filed as derived are holding truth.

---

An object store and a time-series store are usually where a fact first lands — nothing regenerates a photograph or last Tuesday's CPU samples. They are sources of truth wearing the clothes of a derived tier.

The genuinely derived stores are the search index, the vector index, the cache and the warehouse. Anything derived must be rebuildable, must be allowed to lag, and must never be the only copy.



# An object store is the cheapest home for a write-once file.

## Reach for it when

Items are large, written once, fetched by key, and must survive for a decade.

## Walk away when

You need to modify part of an object, or to list and filter by what is inside it.

## The constraint you inherit

Listing is slow and expensive. The index of what you stored belongs somewhere else.



# A cache converts a correctness problem into a timing problem.

## Reach for it when

Reads repeat, the source is expensive, and slightly stale is genuinely acceptable.

## Walk away when

Staleness is unsafe, or the working set is larger than the cache will ever be.

## The constraint you inherit

Invalidation. You now own a second copy whose wrongness is measured in seconds.



# A durable log turns "do it now" into "do it reliably, soon."

## Reach for it when

Producers outpace consumers, or several systems need to see the same events.

## Walk away when

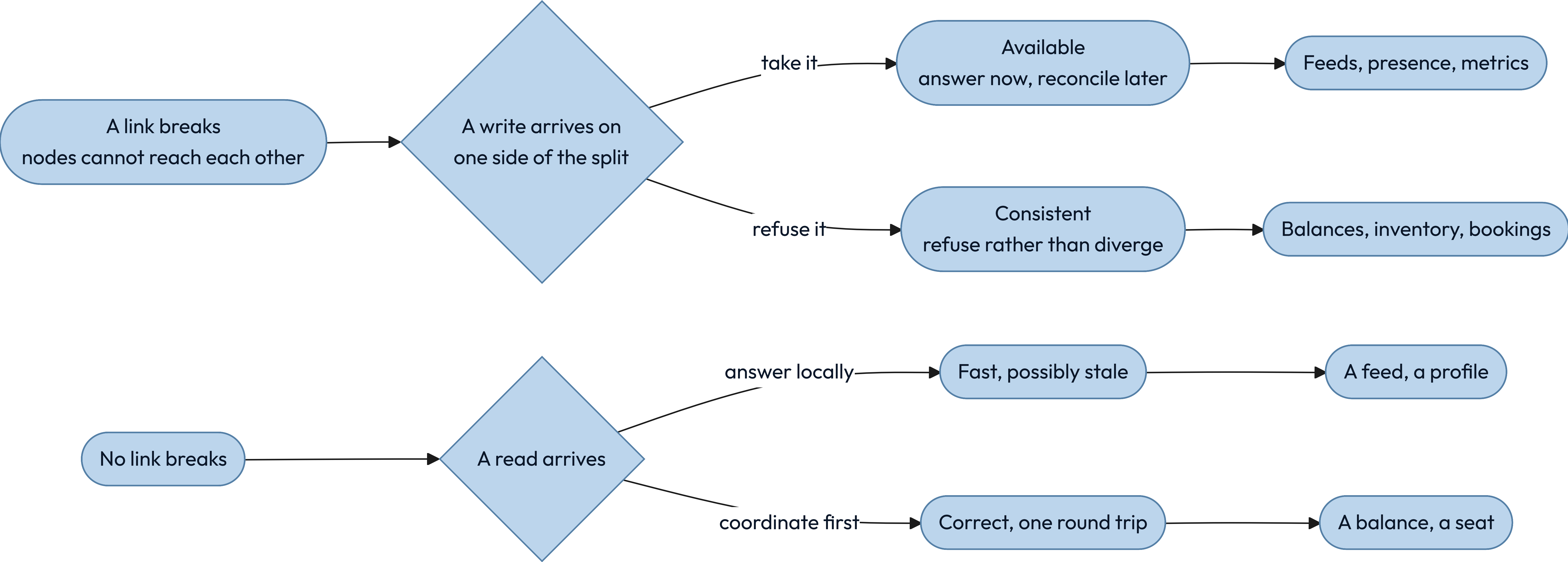
The caller needs the result inside the same request.

## The constraint you inherit

At-least-once delivery. Every consumer must be idempotent or you will charge somebody twice.



# CAP prices a partition. Consistency charges you the rest of the time.





# Three rungs cover almost every argument you will have about consistency.

|    |                  |                                                                                     |
|----|------------------|-------------------------------------------------------------------------------------|
| 01 | Linearizable     | Every read sees the latest write. Costs a coordination round trip, every time.      |
| 02 | Read your writes | You always see your own edits. The least a person who just typed something accepts. |
| 03 | Eventual         | Replicas agree in the end. Cheapest, and correct for feeds, counters and presence.  |



# ACID and BASE answer different questions.

Ask whether coordinating now costs less than reconciling later.

## I ACID

Coordinate first, so the data is never observably wrong. You pay in latency and in how far you can partition.



## II BASE

Diverge now, converge later. You pay in application code, because every reader must tolerate stale state.



# An index is a second copy of your data, sorted for exactly one question.

Indexes make reads fast by making writes slower and storage larger. Each one is a promise to maintain a sorted structure on every single insert, update and delete, forever.

## THE TELL

You can name the exact query each index exists to serve, out loud, right now.

## THE BILL ARRIVES ON WRITES

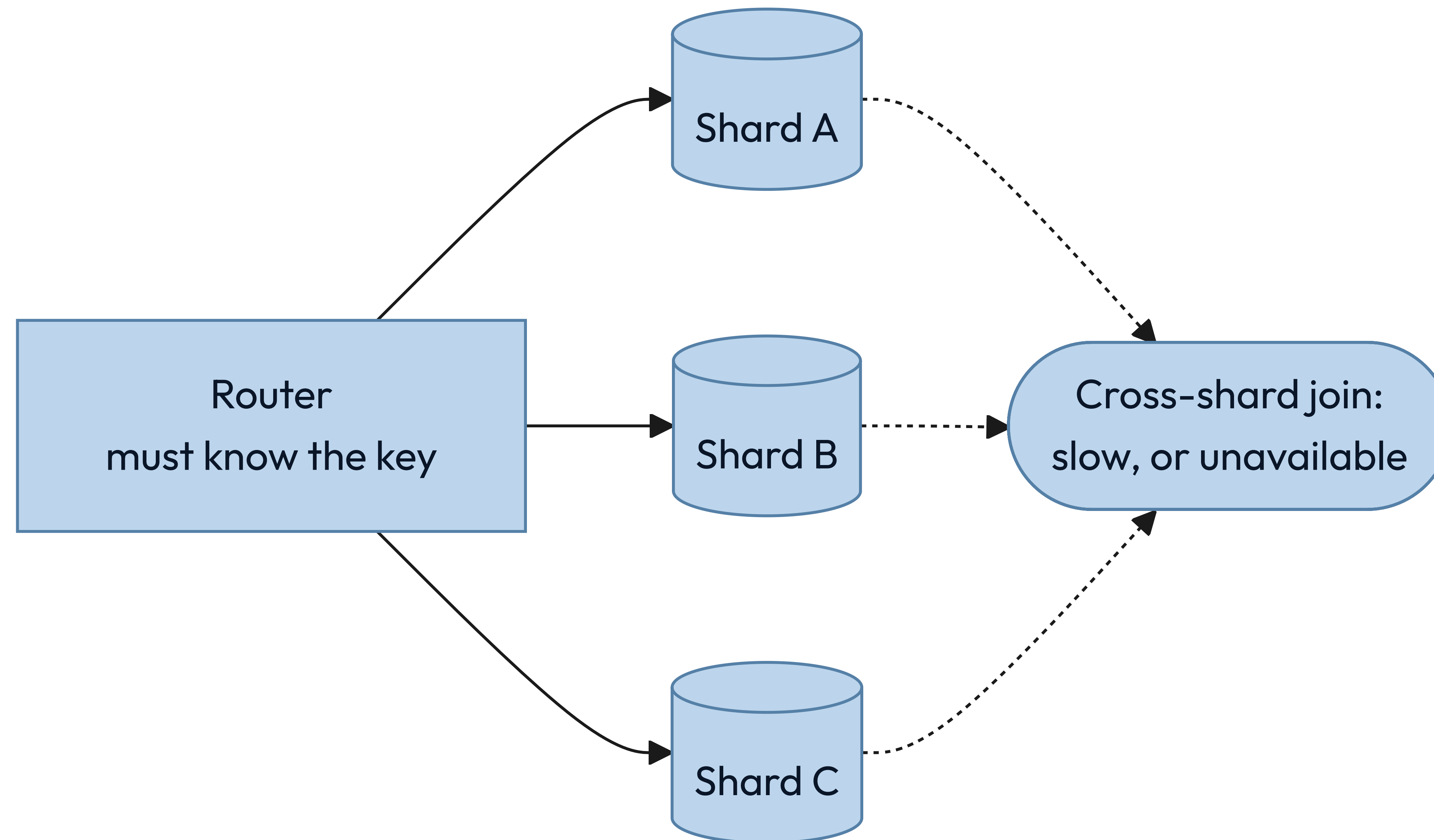
Five indexes mean five extra structures touched per insert, not one.

## HOW MANY ROWS IT REMOVES, NOT HOW MANY VALUES IT HAS

Three evenly spread values get ignored. Three where one appears in 0.1 percent do not.



## Sharding buys throughput by giving up the questions that cross shards.





# The shard key is the decision you cannot undo cheaply.

## By hash of the key

Spreads load evenly and destroys range queries. The safe default.

## By range

Keeps ranges fast and invites a hot spot on the newest range.

## By tenant or region

Matches both the access pattern and the law, until one tenant grows enormous.

### KEY INSIGHT

Name the query that will now cross shards. If you cannot, you have not chosen a key — you have chosen a hash.



# Two machines never agree on the time, so never let a clock decide an order.

## Reach for it when

Anything is sorted, paged or deduplicated. A sortable id carries time and writer in one value.

## Walk away when

Never, once a second writer exists. Milliseconds of clock skew both reorder and collide.

## The constraint you inherit

An id you sort by can never change, and every page cursor comes to depend on it.



# Nothing running lets you change a column in a single step.

STEP 01

**Expand**

Add the new field.  
Nothing reads it yet,  
and old code and  
new code both still  
work.

STEP 02

**Backfill**

Fill it for the rows  
that already exist, in  
batches, with a job  
you can stop and  
restart.

STEP 03

**Cutover**

Move reads to the  
new field. Keep  
writing both, so  
going back costs  
nothing.

STEP 04

**Contract**

Delete the old field,  
and only once  
nothing has read it  
for long enough to be  
sure.



# A deploy is never atomic, so every change has to tolerate the version beside it.

---

For the minutes or hours a rollout takes, two versions of your code run against one store. Both have to work. That is the entire reason for the four steps: each one is safe while the step before it is still live.

The same rule governs an API other people call. Add fields, never repurpose them. Make anything new optional. Remove nothing until you can show that nobody calls it — and if you cannot show that, you have not earned the right to remove it.



# A delete is a fan-out, and it has to reach every copy you ever made.

## The row itself

A tombstone, not a gap. Replicas have to learn it is gone, not merely fail to see it.

## Every derived copy

Caches, feeds, search indexes, aggregates. Each holds its own copy and each needs telling.

## The backups

You cannot rewrite a backup. Set a retention window and let the copy expire instead.

## The bytes at the edge

A CDN serves what it cached. Purging is eventual, so signed URLs need short lives.

## KEY INSIGHT

Design the delete when you design the write. Retrofitting one across six stores is a quarter of somebody's year.



# Four sentences should hold in any data design. Which does yours break?

---

01

## **Exactly one source of truth per fact**

Every other copy is derived, and is labeled as derived where people can see it.

02

## **Every derived store can be rebuilt**

If the search index burns down, a job restores it from the source, unattended.

03

## **Every consumer of a queue is idempotent**

At-least-once delivery is the only delivery anyone actually gets.

04

## **Every write path states its consistency level**

"Whatever the database does by default" is not a stated level.

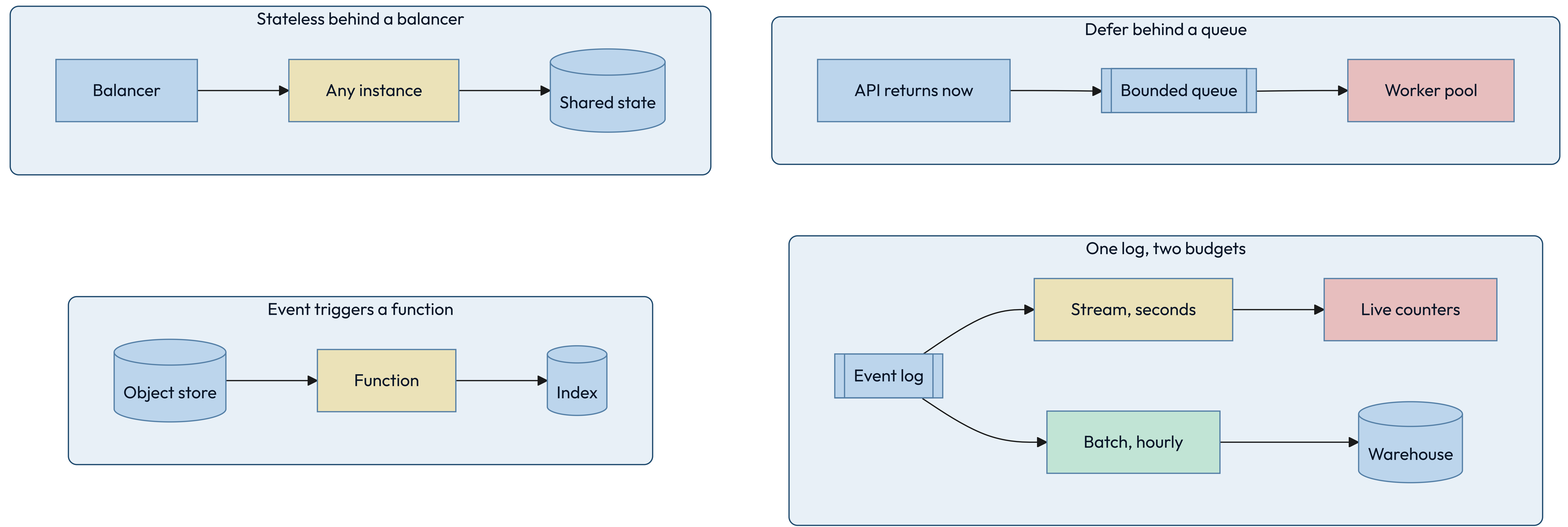


COMPUTE

Choosing compute is  
choosing how much of  
the machine you still own.



# Each of these hands the machine to somebody else.





# A virtual machine is what you reach for when the process outlives the request.

## Reach for it when

The workload runs continuously, holds state in memory, or needs unusual kernel settings.

## Walk away when

Traffic is spiky and idle machines start to dominate the bill.

## The constraint you inherit

Patching, capacity planning, and the gap between staging and production are now yours.



# A container makes the deployable unit identical everywhere it runs.

## Reach for it when

You run many services, deploy often, and want one packaging story across all of them.

## Walk away when

You run one small service. The scheduler that places containers on machines costs more than it saves at that size.

## The constraint you inherit

The orchestrator is now infrastructure, with its own failure modes and its own pager.



# A function is compute you rent by the millisecond, so idle costs you nothing.

## Reach for it when

Traffic is spiky or rare, the work is short, and per-request isolation is welcome.

## Walk away when

Runs are long, or steady traffic makes per-request pricing more expensive than a machine.

## The constraint you inherit

Cold starts, unless you pay to keep instances warm — which is paying for idle again.



# Statelessness is what makes a machine replaceable.

## A STATEFUL INSTANCE

It holds something no other instance has: a session, a lock, a warm cache, an open connection. Scaling means moving that state, and a crash means losing it.



## A STATELESS INSTANCE

Any instance serves any request because the state lives somewhere else. Scaling is arithmetic and failure is a routing change.



# Anything you run should satisfy these four. Which does yours break?

---

01

**Any instance can be killed without a customer noticing**

If that is false, you have state you have not named yet.

02

**Deployments are reversible**

A rollback is a routine operation, not an incident response.

03

**Capacity is a number somebody owns**

Not "it autoscales" — the ceiling, the cost, and who gets paged at it.

04

**Startup does not depend on startup order**

Services retry into each other instead of requiring a sequence.

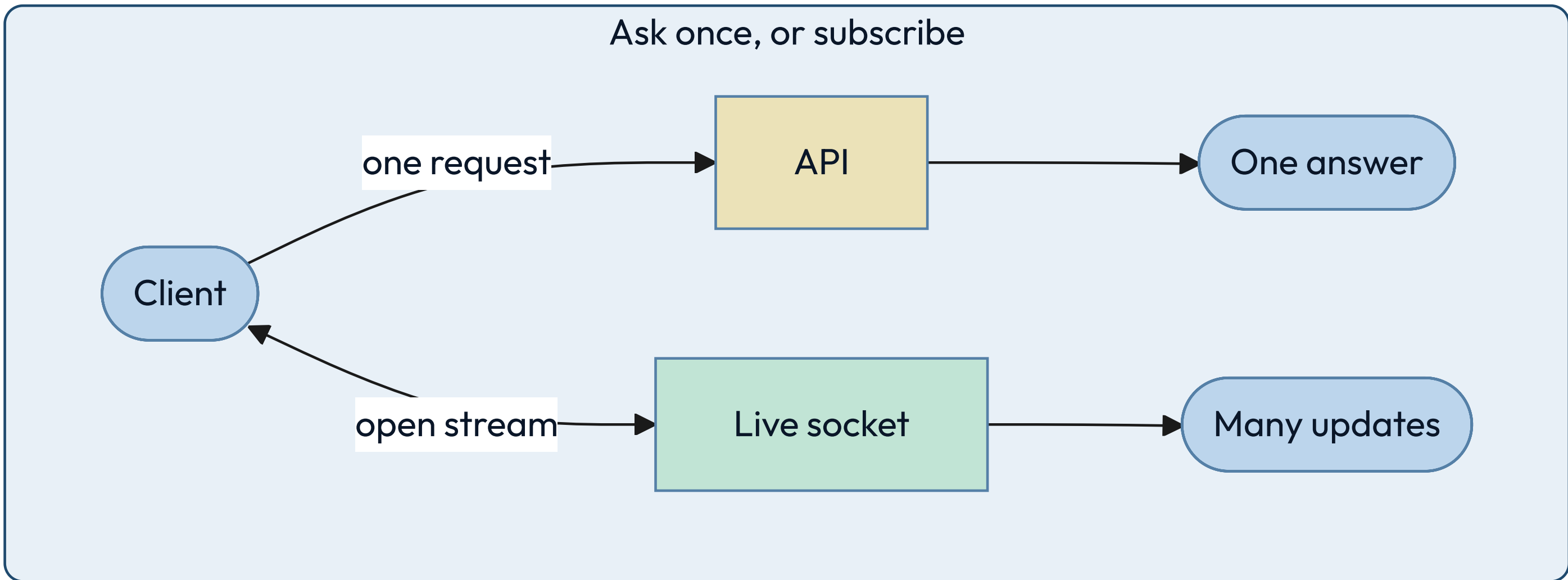
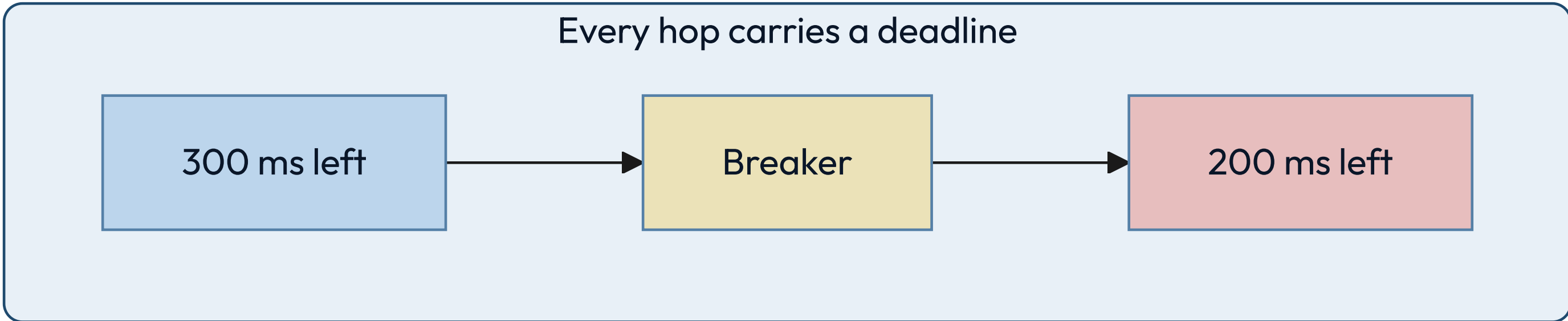
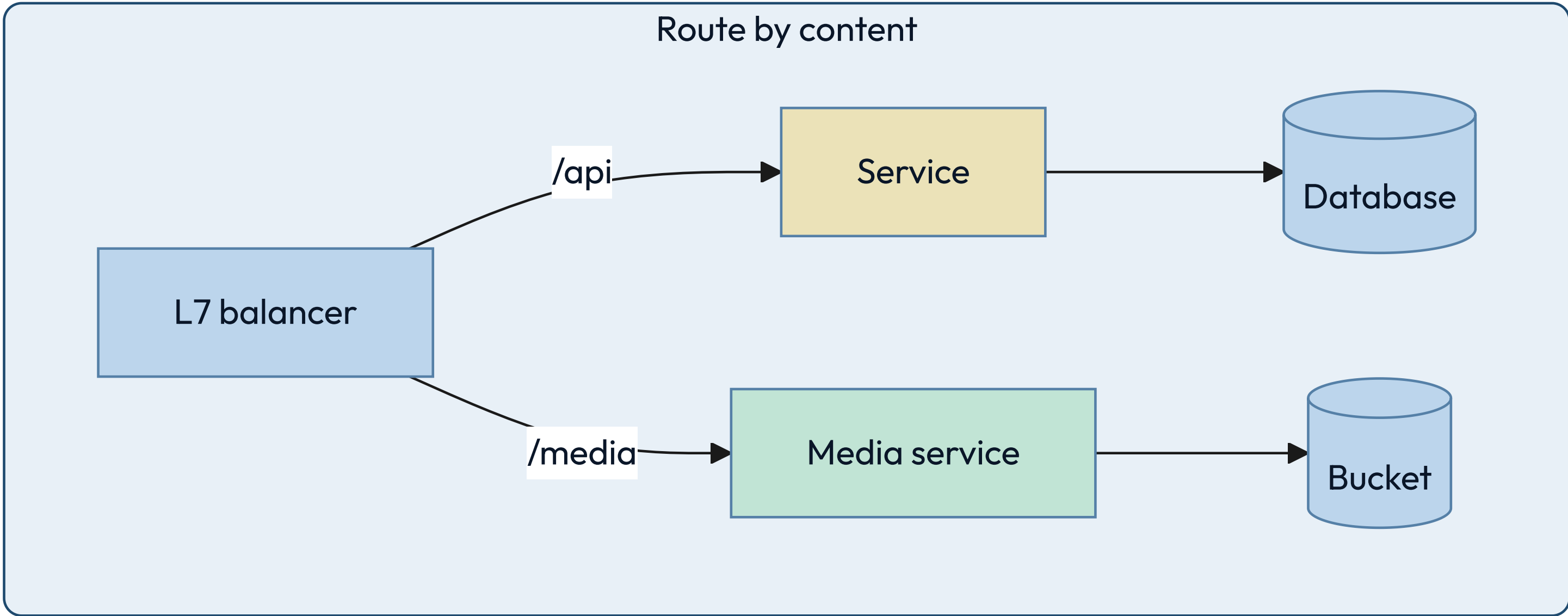
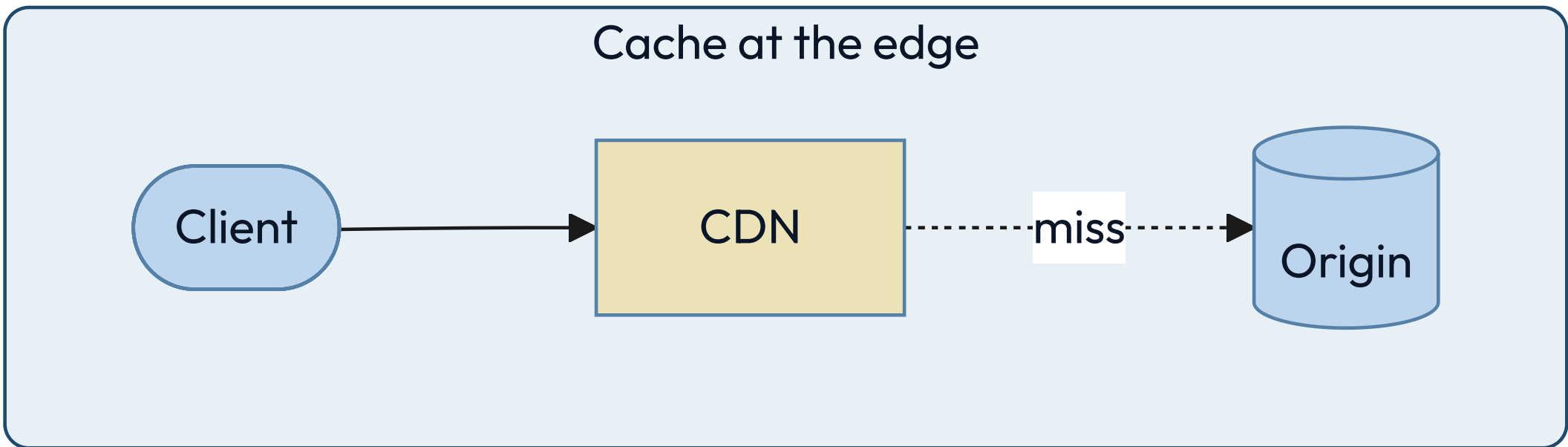


NETWORK

**Distance is the one cost  
you cannot optimize away.**

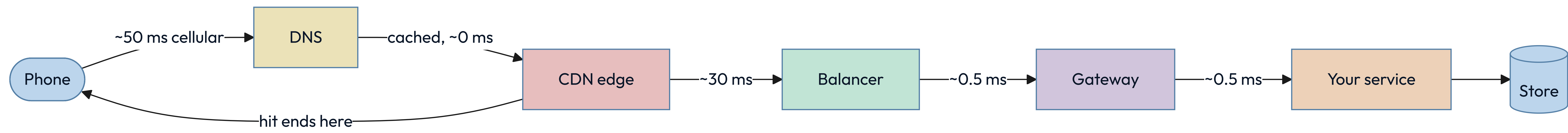


# Distance costs you differently in each of these.





# Four hops separate a tap from your service, each spending budget.





# Three numbers settle most latency arguments before they start.

|    |                            |        |
|----|----------------------------|--------|
| 01 | Memory read                | 100 ns |
| 02 | Datacenter hop             | 0.5 ms |
| 03 | Cross-continent round trip | 150 ms |



WHY THAT NUMBER IS FINAL

# Light in fiber covers about 200 kilometers per millisecond.

---

Nothing changes that. The measured 150 milliseconds is well above the straight-line floor, because packets do not travel in straight lines and every hop queues.

A design that needs three sequential intercontinental round trips has spent half a second before it executes an instruction. Replication, caching and edge delivery all exist to buy that distance back, and none of them makes it free.



# A CDN moves bytes closer to readers, and ends the connection there too.

## Reach for it when

The content is large, popular, and identical for many people.

## Walk away when

Nothing is shared and nothing is far. A personal response still wins the handshake back.

## The constraint you inherit

A second copy with its own staleness. Version immutable URLs; purge the rest, and time the purge.



# A request with no timeout is a resource leak waiting for a bad afternoon.

Every waiting request holds a connection, a thread and some memory. Under a slow dependency, unbounded waits turn one struggling service into a queue of stuck callers — which is the shape of the page that pulled Maya in at twenty to eleven.

## WHAT GOOD LOOKS LIKE

Every outbound call has a deadline, and the deadline shrinks as it propagates.

## RETRIES NEED A BUDGET

Retrying without a cap turns a brief failure into a flood you built yourself.

## BACKOFF MUST BE RANDOM

Synchronized retries arrive together and rebuild the spike you just survived.



# A call that leaves your process owes you these four.

---

01

## **Every remote call has a deadline**

Inherited from the caller, and always shorter than the caller's own.

02

## **Every retry has a budget and jitter**

Bounded attempts, randomized delays, and a breaker when the target is down.

03

## **Every write is idempotent or keyed**

The network will deliver your request twice. Decide now what that means.

04

## **Distance appears in the design**

Round trips between regions are counted on purpose, not discovered in production.



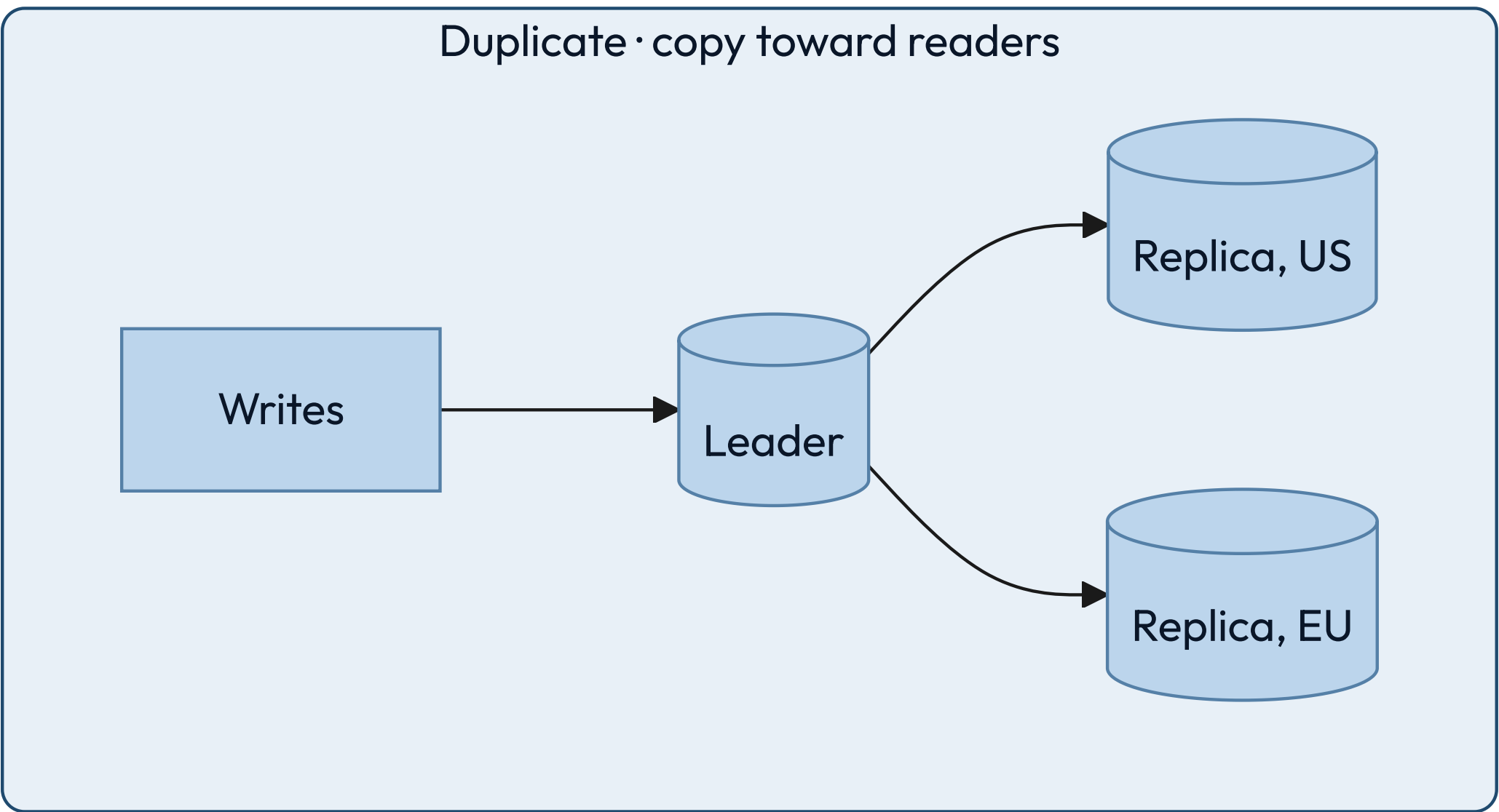
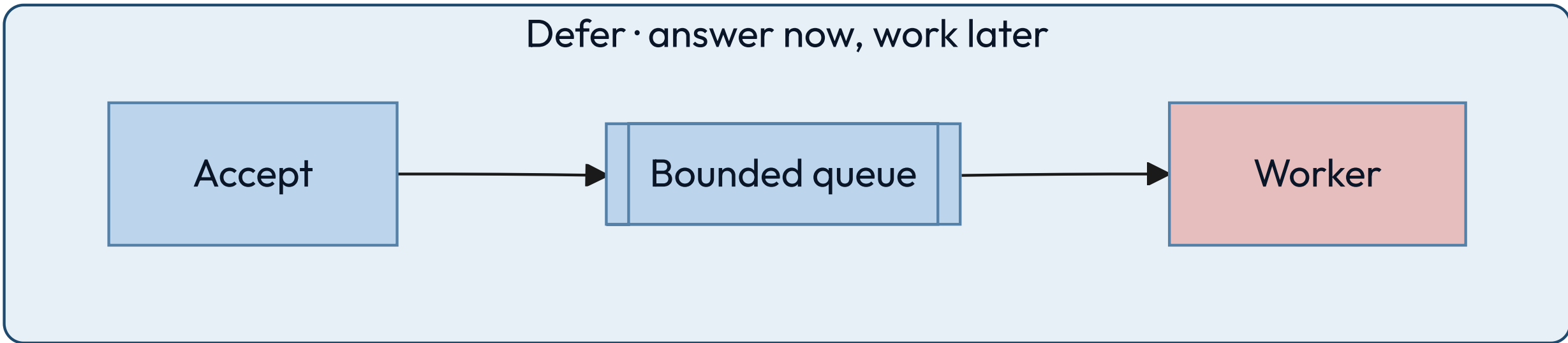
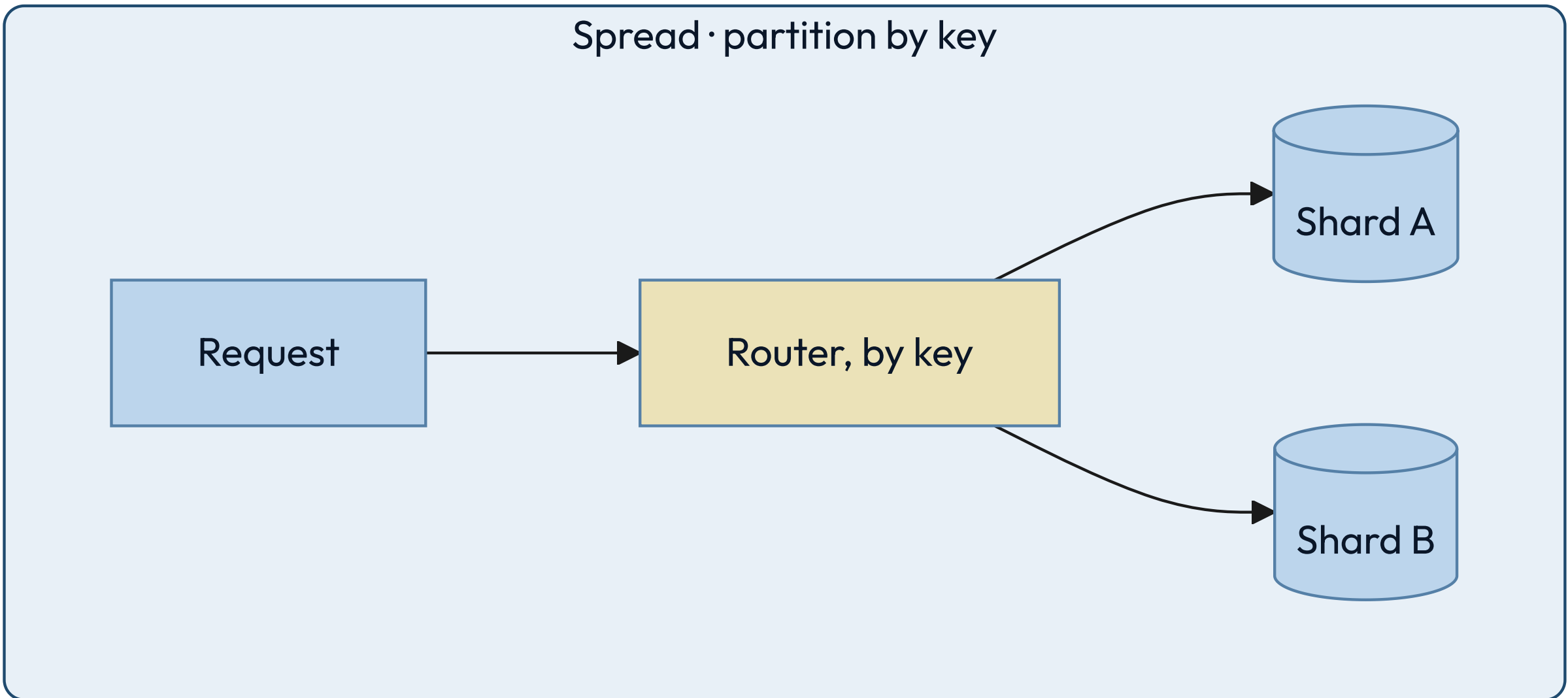
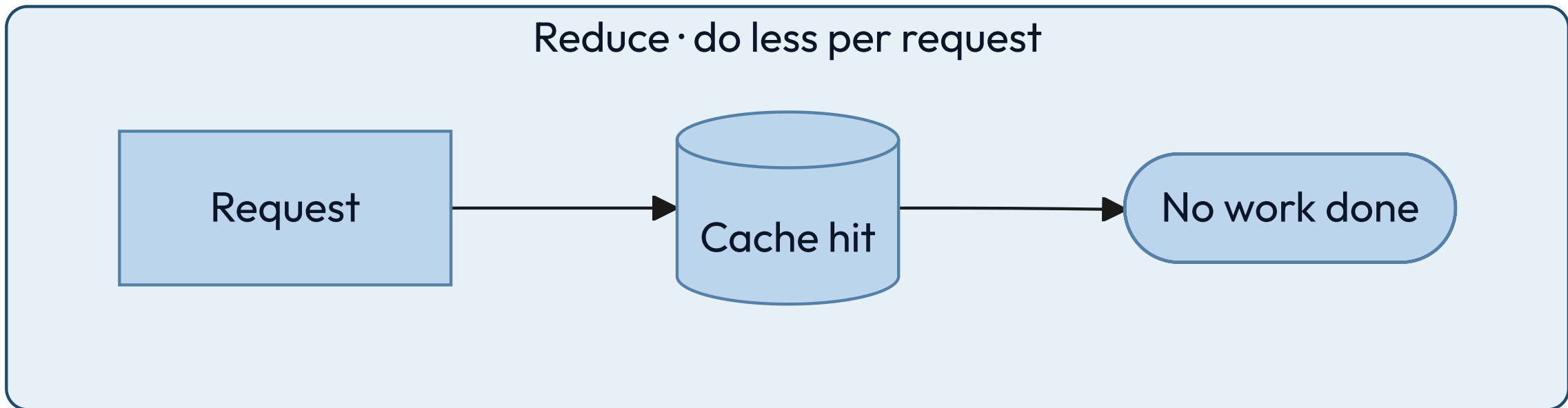
SCALE

Every scaling change  
is one of four moves.



SCALE KIT • THE PATTERNS

# The four moves, and what each one costs you.





# Little's law ties concurrency, throughput and latency together.

$$L = \lambda W$$

WHERE

- $L$  — requests in flight at once
- $\lambda$  — arrivals per second
- $W$  — time each one spends inside



## USING IT

# The formula sizes your thread pool before anyone guesses.

---

At 2,000 requests per second averaging 50 milliseconds each, 100 requests are in flight at any moment. That is your minimum concurrency, and no tuning makes it smaller while the other two numbers hold.

It also explains the outage, and which way it goes depends on the pool. Leave it unbounded and tripled latency triples the requests in flight, until memory runs out. Bound it at 100 and the concurrency cannot rise, so throughput falls instead —  $100 / 0.15s$ , about 667 a second against the 2,000 arriving — and the queue in front grows without limit. That second failure is why every queue needs a ceiling. Find your own numbers: arrivals on the load balancer, duration in the handler.



# The average request is a fiction, and your users live in the tail.

A page that makes 100 parallel calls waits for the slowest one. Give each call a one-percent chance of being slow. Then 63 percent of pages hit at least one slow call. That is  $1 - 0.99^{100}$ , and you can redo it on a napkin.

## THE CHECK

Percentiles for what a person experiences, means for what a fleet must total. Report the 99th per dependency; keep the mean for capacity, never for latency.

## FAN-OUT AMPLIFIES IT

More parallel calls turn a rare slow response into a common slow page.

## HEDGING BUYS IT BACK, ON A BUDGET

Send a duplicate after the 95th percentile and take whichever answers first. A hedge is a second request, so cap it — a few percent of traffic. Unbudgeted, it is the reinforcing loop again, arriving exactly when you are already slow.



# Each caching pattern owns a different failure.

## Cache-aside

The app fills the cache on a miss. Simple, and it stampedes on a cold key unless one reader fills it while the rest wait on that one fill.

## Read-through

The cache fetches for you. Cleaner code, and now the cache is on the critical path.

## Write-through

Write both together. Always fresh, and every write pays the cache's latency.

## Write-behind

Write the cache, flush later. Fastest writes, and a crash loses them.

## KEY INSIGHT

Choose by which failure you can survive, not by which pattern reads best in code.



# Idempotency is what makes a retry safe, and retries are not optional.

Networks duplicate, clients retry, queues redeliver. The only question is whether the second delivery is harmless or charges somebody twice.

## THE RULE

Every mutating endpoint takes a client-supplied key and deduplicates on it.

## THE KEY COMES FROM THE CALLER

Generated before the first attempt, reused on every retry of that same intent.

## STORE THE RESULT, NOT THE FACT

A repeat returns the original answer, not an error saying it already happened.



# Check these four before you call anything scalable.

---

01

## **The bottleneck is named and measured**

Not suspected. A number, a graph, and the resource it belongs to.

02

## **Every queue is bounded**

An unbounded queue turns a throughput problem into a memory outage.

03

## **Admission control sheds load before the system collapses**

A balancing loop from Part one: reject at the edge, not everywhere at once.

04

## **Adding a machine is routine**

No manual steps, no rebalancing outage, no cold-cache stampede.



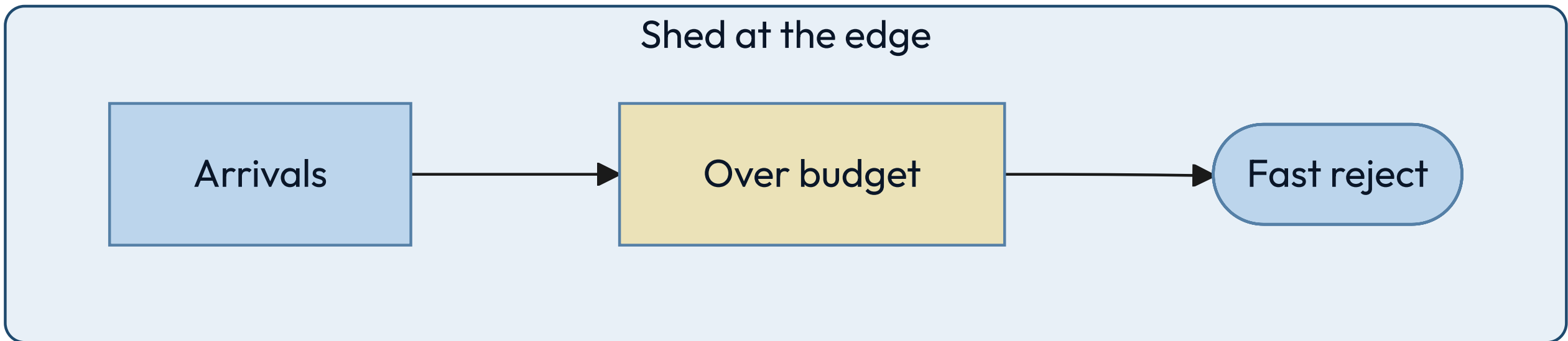
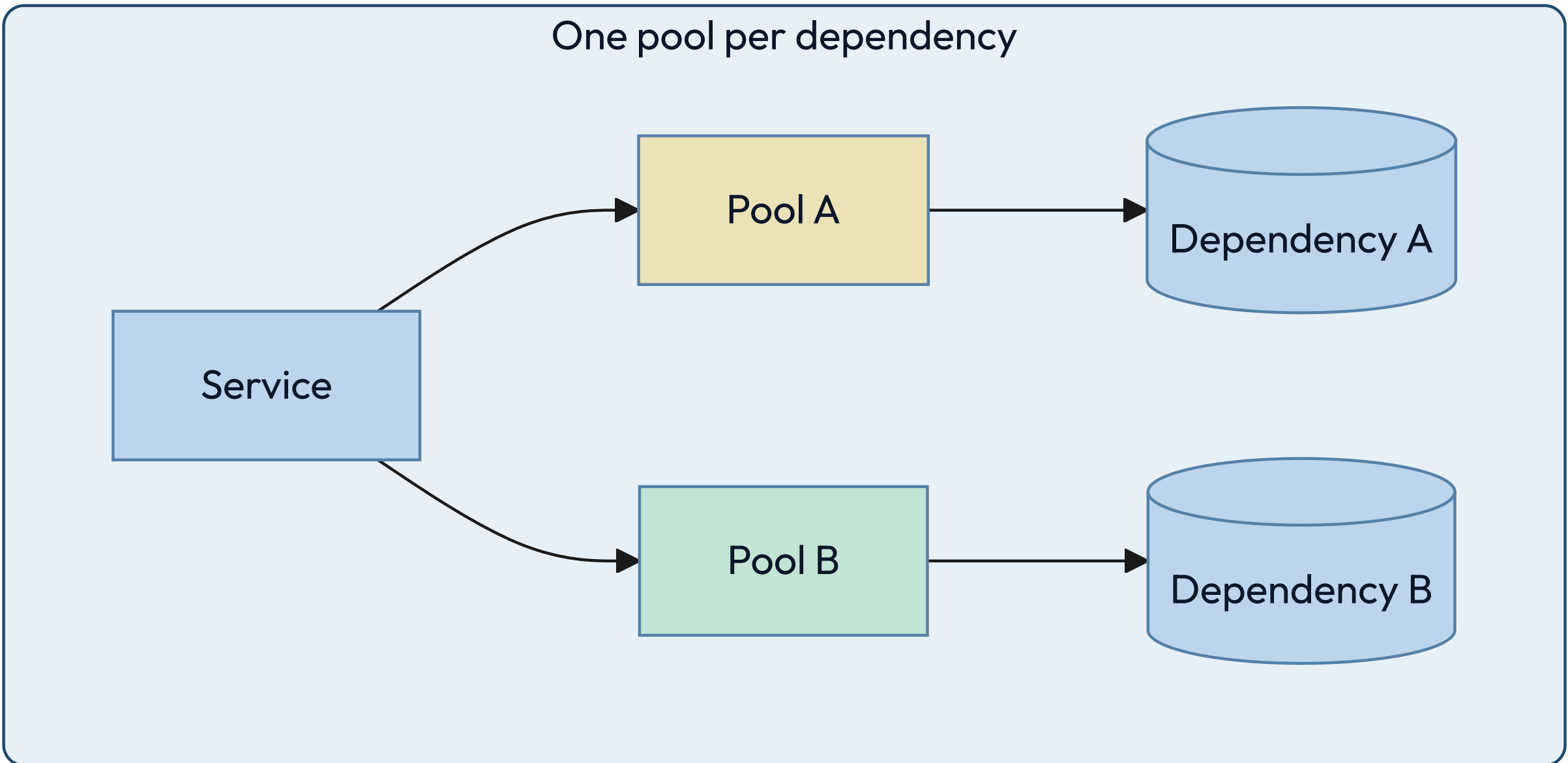
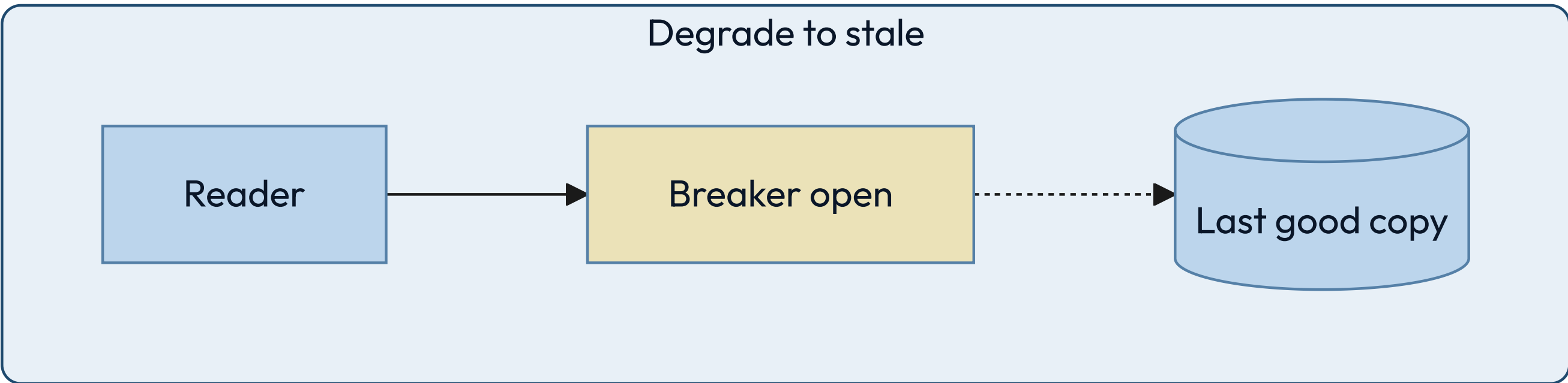
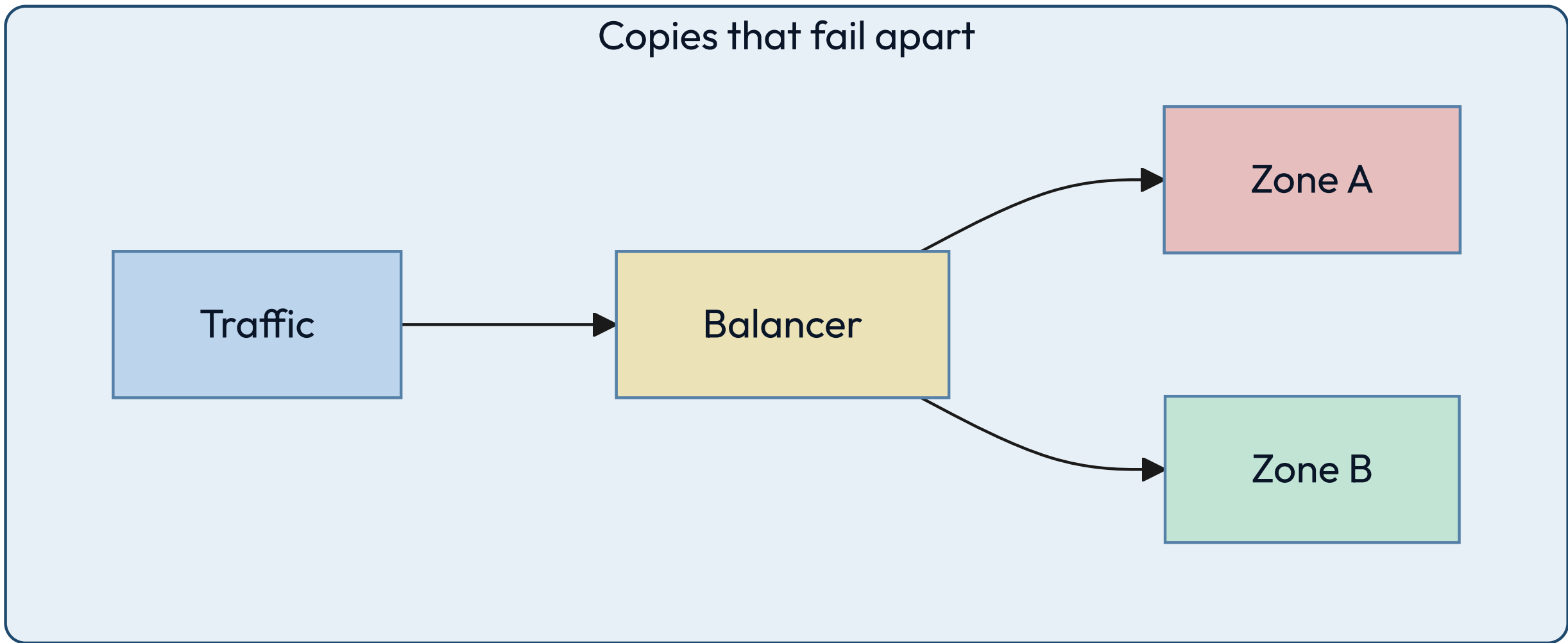
RELIABILITY

**Failure is the environment,  
not the exception.**



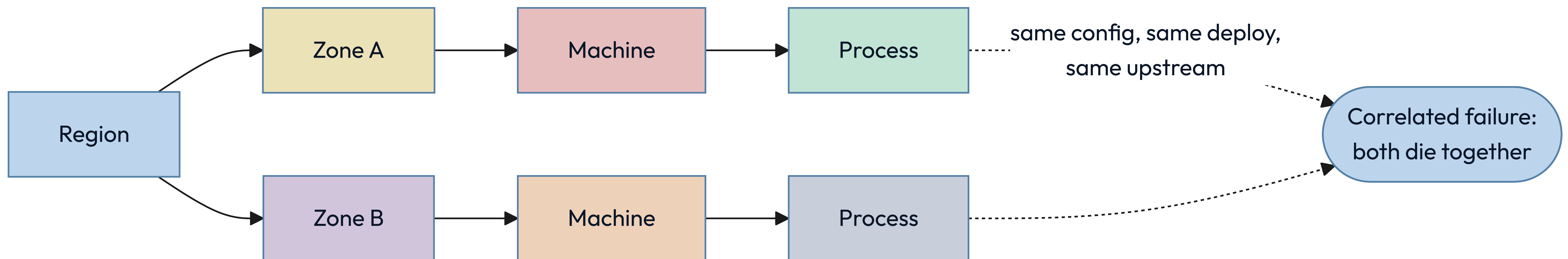
RELIABILITY KIT • THE PATTERNS

Each of these keeps one failure from becoming every failure.





## Redundancy only helps when the copies can fail apart.





# Four mechanisms keep a slow dependency from taking the building.

## STEP 01

### Timeout

Bound the wait, so a slow callee cannot hold your resources.



## STEP 02

### Retry with backoff

Recover from a blip, with a budget and jitter. Retries without them are a reinforcing loop.



## STEP 03

### Circuit breaker

Stop calling something that is clearly down, and let it recover.



## STEP 04

### Bulkhead

Give each dependency its own pool, so one queue cannot drain them all.



# Four terms decide what you are allowed to ship this week.

# 01 SLI

The measurement: the share of requests served under 300 milliseconds.

## 02 SLO

Your internal target for it, such as 99.9 percent over 28 days.

## 03 SLA

The external promise, with money attached, and always looser than the SLO.

## 04 Budget

## What the objective lets you spend. Exhausted, feature work stops.



# An availability target is a budget that shrinks fast.

|    |                |                      |
|----|----------------|----------------------|
| 01 | 99 percent     | 3.65 days per year   |
| 02 | 99.9 percent   | 8.8 hours per year   |
| 03 | 99.99 percent  | 53 minutes per year  |
| 04 | 99.999 percent | 5.3 minutes per year |



**Past three nines the humans stop being fast enough.**

Fifty-three minutes a year is about one incident with a page, a login, a look at a dashboard and a decision. So four nines does not mean a faster on-call engineer. It means automatic failover and automatic rollback, because a person is no longer in the loop.

Each nine roughly multiplies the cost. Pick the number the business actually needs, and write down what you are choosing not to buy.



# Three signals answer three different questions. None replaces another.

## Metrics

Cheap, aggregate, always on.  
They tell you that something is wrong.

## Logs

Detailed, expensive, one event at a time. They tell you what happened in one case.

## Traces

One request across every service. They tell you where the time actually went.

### KEY INSIGHT

If you cannot answer "which dependency is slow" inside a minute, you have metrics, not observability.



# A well-designed system gets worse in an order somebody chose.

Under pressure something has to give. Either you decided in advance which features degrade, or the system decides at random and drops the one that takes money.

## THE RULE

Every feature has a stated tier, and the lowest tier fails first by design.

## READ PATHS OUTLIVE WRITE PATHS

Serving something slightly stale beats serving an error page, for almost every product.

## THE FALLBACK IS EXERCISED

An untested degraded mode is untested code running in your worst hour.



# Trust nothing in production until these four hold.

---

01

**Every dependency has a defined failure behavior**

Written down: degrade, queue, or fail fast. Never "we will see."

02

**Redundant copies fail independently**

Different zones, different deploys, different upstreams. Otherwise it is one copy.

03

**Recovery is practiced**

Restores, failovers and rollbacks happen on a schedule, not for the first time at 3am.

04

**The system tells you before a user does**

Alerts fire on the indicator, never on the complaint.

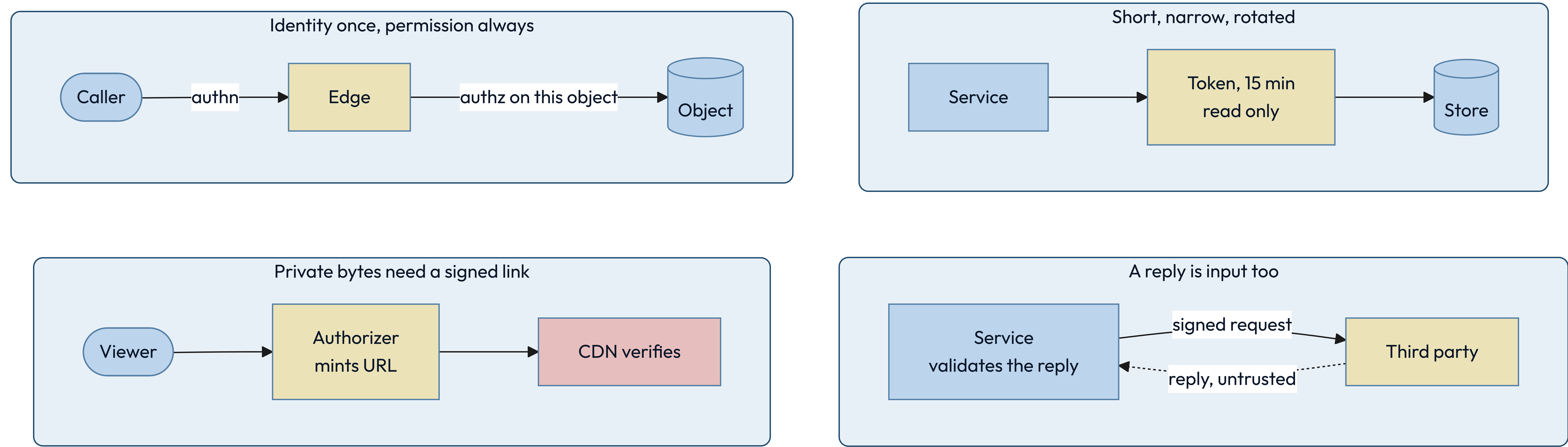


SECURITY

Assume the boundary  
is already crossed.

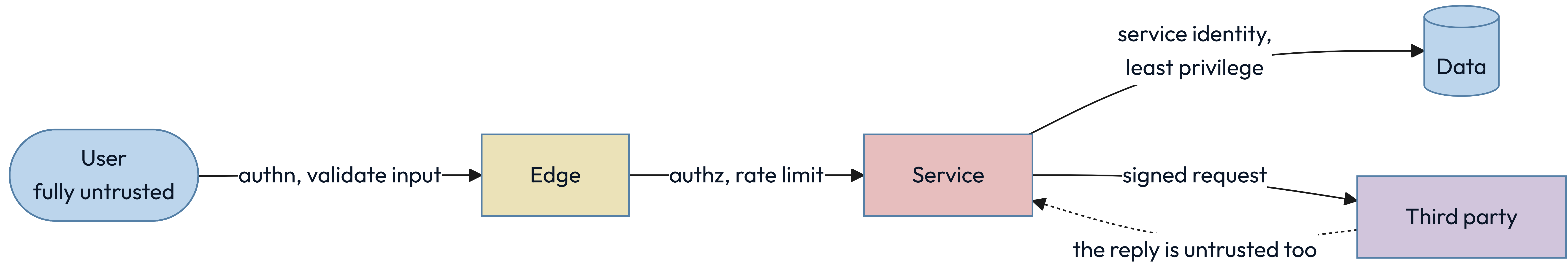


All four of these assume somebody is already inside.





# Every arrow that crosses a trust boundary needs a check on the far side.





# Authentication asks who you are. Authorization asks what you may touch.

## AUTHENTICATION

Establishes identity once, at the edge, and hands down a claim anyone downstream can verify. Getting it wrong lets the wrong person in.



## AUTHORIZATION

Decides, on every single request, whether this identity may touch this specific object. Getting it wrong lets the right person read somebody else's data.



# Least privilege is measured by what one stolen credential can reach.

Assume any single credential eventually leaks. The design question is not whether that happens; it is how far the person holding it can travel, and for how long.

## THE CHECK

You can say in one sentence what each service account could reach if stolen.

## SCOPE IT AND EXPIRE IT

Narrow permissions, short lifetimes and automatic rotation beat a careful human.

## SEPARATE READ FROM WRITE

Most services need one or the other. Almost none needs to delete.



# Encryption has three states, and a fourth thing that outranks all of them.

# 01 Transit

TLS everywhere, including between your own services. Networks are untrusted.

## 02 Rest

Disk and backup encryption. Stops a stolen device, not a stolen credential.

## 03 Use

Decrypted in memory to be processed. The hardest state, and the one attacked.

## 04 Keys

A managed store or an HSM, rotated on a schedule, never in the repository.



# Six questions find most of the holes before somebody else does.

- ☐ Can someone pretend to be another identity? spoofing
- ☐ Can data be changed in transit or at rest? tampering
- ☐ Can an actor deny having done something? repudiation
- ☐ Can private data reach the wrong reader? disclosure
- ☐ Can one caller exhaust a shared resource? denial
- ☐ Can a caller gain rights nobody granted? elevation



# Most of the code you ship, you did not write and have not read.

---

Maya's Tuesday stopped when a package registry went down. That same registry is also the shortest path into her build: whoever can publish a version she installs can run their code on her machine, and then inside her deploy.

Pin exact versions and commit the lock file, so the build you tested is the build that ships. Keep an inventory of what you actually depend on, including everything your dependencies pulled in behind you. Take updates on a schedule you chose rather than the moment they appear, and read the diff of anything that touches a credential, a network call, or a build step.



# An authorized caller can still take the whole system down.

## Reach for it when

Anything is shared: a database, a queue, a paid third party, an image decoder.

## Walk away when

Never entirely. Put the limit where an ordinary caller will never meet it.

## The constraint you inherit

A limit needs an identity to count against. Use the account, the key and the address.



# Sign your name to a design only when these four are true.

---

01

**Every request is authorized against the specific object**

Not the endpoint. Object-level checks are where the breaches happen.

02

**Secrets never enter the repository or a log line**

They live in a managed store, are injected at runtime, and they rotate.

03

**All input is untrusted, including from your own services**

A compromised internal caller is the ordinary case, not the exotic one.

04

**Every privileged action is attributable**

An immutable record of who, what, when, and from where.



# 05

---

## PART FIVE

Now we design the thing Maya opened while the train sat.



Eight of these you fill in before the first box. The ninth you cannot know yet.

|               |                                                                                                    |
|---------------|----------------------------------------------------------------------------------------------------|
| Protagonist   | A reader on a phone. Twelve opens a day, on cellular.                                              |
| Antagonist    | The shape of the follow graph – a tail at $5 \times 10^8$ edges.                                   |
| Purpose       | A fresh, personal, ranked page in under a second.                                                  |
| Boundary      | In: feed, posts, edges, media. Out: the phone, the network, the law.                               |
| Environment   | Evening peak $\sim 2.5\times$ the mean, partitions, duplicate deliveries, people who want in.      |
| Constraints   | 200 ms p99 • 60 reads per write • media durable forever                                            |
| Invariants    | A post reaches eligible followers • media never lost • a like counts once                          |
| Bottleneck    | Suspected: filling in each post. Confirmed at $\sim 80\text{M}$ reads/s once the read path exists. |
| Solution type | Scaled. Not an MVP, not optimal.                                                                   |



# Four operations carry the entire product.

---

01

## Post a photo

Upload media, attach a caption, publish it to the people who follow you.

02

## Follow an account

Build the directed graph that decides whose posts you are eligible to see.

03

## Read a feed

A ranked, paginated page of recent posts from the accounts you follow.

04

## React

Like and comment, with counts visible on every post.



# Four guarantees decide the architecture more than the features do.

---

01

**The feed serves in under 200 milliseconds**

Server-side, 99th percentile, measured from the reader's own region.

02

**Reads dominate writes by about sixty to one**

At the API. Push fan-out inverts it underneath: 330K feed writes a second against 70K reads.

03

**Posts, media and the graph are durable forever**

Lose any of the three and nothing rebuilds them. Only derived data rebuilds.

04

**Eventual consistency is fine on the feed**

A late post and a lagging count are fine. A blank count is not.



# Two assumptions and some division give you every number that matters.

---

Assume 500 million daily users opening the feed twelve times a day, and 100 million photos posted a day. The rest is division. Twelve opens each, times five hundred million people, over the seconds in a day: about **70K** reads a second. A hundred million posts over the same day: **1.2K** writes. At two megabytes each: **200 TB** a day. The ratio lands near **60:1**.

One of those numbers does less work than it looks. Change the 500 million, holding opens and posts per person fixed, and the ratio does not move at all.



**Four numbers decide something here. The 500 million users decide nothing.**

|    |                                |                                                                       |
|----|--------------------------------|-----------------------------------------------------------------------|
| 01 | <b>Writes · 1.2 K/s</b>        | One machine could take these. The write path was never the hard part. |
| 02 | <b>Feed reads · 70 K/s</b>     | A fleet, and the number everyone sizes from. It is not the real one.  |
| 03 | <b>Media · 200 TB a day</b>    | An object store, not a database. Settled right here.                  |
| 04 | <b>Followers · up to 500 M</b> | The only number with no ceiling. Two read paths come from this.       |



## YOUR TURN

**Redo it for a smaller product before you turn the page.**

Fifty million daily users, opening the app four times a day, posting two million photos. Work out reads per second, writes per second, and the ratio between them.

Do it now, on paper, in under a minute. Cover the next slide until you have one.

The point is not the answer; it is that you can produce one at all, in the meeting where it is being decided.



## THE ANSWER

# The ratio moved, and the reason it moved is the lesson.

$50\text{M} \times 4 = 200\text{M reads/day} \div 86,400 \approx 2.3\text{K reads/s}$  ·  $2\text{M} \div 86,400 \approx 23 \text{ writes/s}$  ·  $\text{ratio} \approx 100:1$

---

A thirtieth of the read traffic, and the ratio went from 60:1 to 100:1. It moved because you changed how often each person posts, not just how many people there are. The ratio is opens per user divided by posts per user: it survives being wrong about population, never about behavior.



# A daily average is not a capacity number, and two multipliers separate them.

---

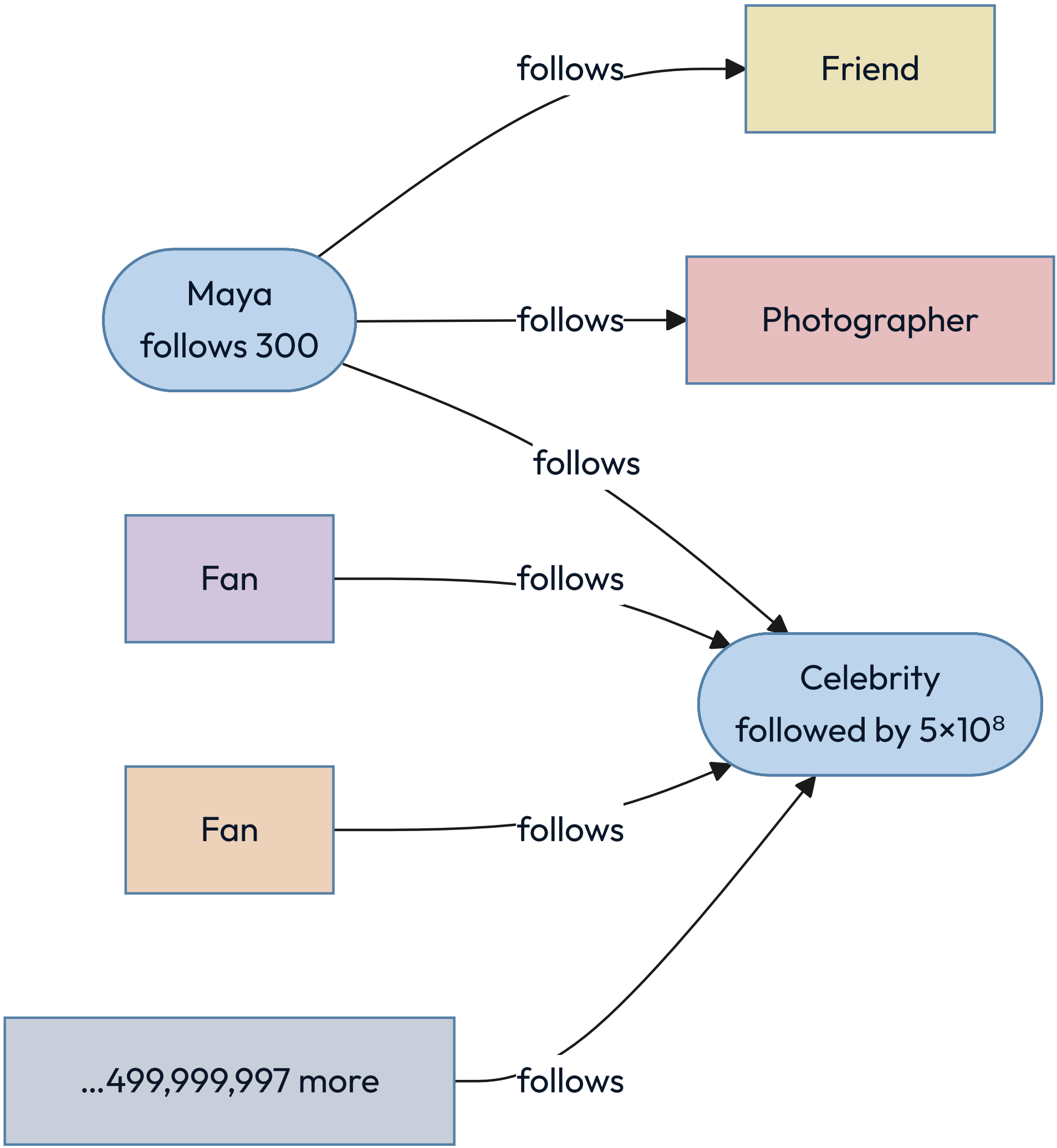
Seventy thousand a second is the average across a whole day, and it counts one page per open. You size a fleet against neither.

An open is a session: the reader scrolls, and each screenful is another fetch. Call it six. Traffic is not flat either — the evening peak runs a few times the daily mean. Call it two and a half.

`70K × 6 × 2.5 ≈ 1M page-fetches a second.` Size against that, and put both multipliers on the slide beside it. They are the two numbers most often left out of an estimate, and leaving them out is how a fleet gets built an order of magnitude too small.



# The follow graph is directed, and the two directions are not the same size.



★ The number of accounts you follow is capped at 7,500. The number who follow you is capped by nothing. Every hard problem in this design sits on that second number.



# Your following list is capped. Your followers list is capped by nothing.

Instagram limits how many accounts you may follow to seven and a half thousand. Nobody limits how many people may follow you. An account on the far end of that second number — hundreds of millions of followers — is called a **super node**, and every hard problem here comes from one.

## FOLLOWING, PER PERSON

*At most 7,500. A read of that list is one partition and a sorted range.*

## FOLLOWERS, PER PERSON

About 150 for most people. Five hundred million at the top. A million times apart.

## EVERYTHING AFTER THIS

Is a consequence of that asymmetry. Nothing else in the design spans that range.



# The edge is stored twice, because it answers two different questions.

## following

PK (source\_id)

CK (target\_id)

val bucket

"does A follow B?" is a point read

capped at 7,500 rows

## followers

PK (target\_id, bucket)

CK (created\_at DESC, source\_id)

the follower list, newest first

bucket = hash(source) % B(target)

B = clamp(followers / 10\_000, 1, 512)

★ The forward edge is the source of truth, and it stores the bucket it was written into. That one column is what lets an unfollow find the reverse edge years later: `B` has grown since, so recomputing `hash(source) % B` now lands somewhere else and the edge would never be deleted. `B` may only grow — shrink it and you lose edges — and a fan-out worker reads it fresh rather than from the display cache, because a stale, smaller `B` scans too few buckets and silently skips the newest followers. Early edges were written when `B` was smaller, so the low-numbered buckets hold several times the average until `B` reaches its cap.



**Four of these reads are cheap. The fifth is the whole problem.**

| THE READ                    | THE PATH                                       | WHAT IT COSTS                          |
|-----------------------------|------------------------------------------------|----------------------------------------|
| Does A follow B?            | <code>following (A)</code> then <code>B</code> | One point read                         |
| Who does A follow?          | <code>following (A)</code> range               | 7,500 rows, one partition              |
| Who follows B, ordinary B?  | <code>followers (B, 0)</code> range            | ~10 <sup>4</sup> rows, one partition   |
| Follower count              | <code>user_edge_stats</code>                   | One point read, cached                 |
| Who follows B, celebrity B? | <code>followers (B, 0..511)</code>             | 5×10 <sup>8</sup> rows, 512 partitions |



LET US GET THIS WRONG

# Design it the obvious way: when you post, write into every follower's feed.

---

The reader then does almost nothing. One lookup, one page, no merging, no ranking across sources. It is fast, it is simple, and for an account with two hundred followers it is unambiguously correct: two hundred small writes, and every reader gets a page in a millisecond.

Hold that design in your head. Now a single account with five hundred million followers posts one photograph. Before you turn the page, predict what happens.



ONE POST, ONE CELEBRITY

# 25 GB

---

Five hundred million feed inserts at roughly fifty bytes each, from a single API call.

## 500 seconds of queue

Assume the fan-out fleet drains a million inserts a second: eight minutes serving nobody else.

## 330K inserts a second

The platform's ordinary load: 1.2K posts times a mean fan-out near 280. Totals need the mean. The median is far lower and would halve your estimate.

## 25 minutes of backlog

One celebrity post is that much of everyone else's work, arriving at once.



# Writing 25 GB from one call is only the first thing that breaks.

## The fan-out queue

25 GB of inserts from one call, in front of everybody else's posts.

## The hot partition

Unbucketed, those edges are one partition key on one replica set.

## The cache

25 GB of churn evicts the working set of readers who did nothing wrong.

## The thundering herd

One cache key, invalidated on publish, missed by a million readers at once.

### KEY INSIGHT

A skewed key produces a system where one machine is on fire and the rest are idle.



## WHY THE HYBRID IS FORCED

# Neither strategy is wrong. Each one is wrong somewhere.

---

Pushing costs one write per follower, and the follower count is unbounded. Pulling costs one read per followee, and the followee count is capped at seven and a half thousand. So push is cheap exactly where the unbounded side is small, and pull is cheap exactly where the bounded side is what you walk.

There is a second reason, and it is the better one: a celebrity's recent-posts list is written once and read five hundred million times. It is the most cacheable object in the system.



## DECISION

**Push below the threshold, pull above it, and merge at read.**

The two strategies fail at opposite ends of the same graph, so the design uses each one where the other breaks.

## ONE STRATEGY FOR EVERYONE

- Simple to explain, guaranteed to fail at one end. Push drowns on celebrities; pull costs hundreds of reads per page for everyone else.

## SPLIT AT THE THRESHOLD



- Ordinary posts land in a page that is already built; the celebrities she follows are fetched on demand and merged in, so no writer ever fans out to millions. The read costs one lookup plus the celebrity pulls.

## RECOMMENDATION

The threshold is not a preference. It is where two cost curves cross, and the graph draws it.



# Fifty thousand followers is the line, and the honest predicate is a rate.

---

Fifty thousand puts a fraction of a percent of accounts on the pull path. Someone following three hundred typically has ten to thirty above the line — not because celebrities are common, but because the accounts a person chooses to follow are not a random sample of accounts.

Crossing the threshold does not rewrite history. Entries already pushed stay where they are and age out.

The better predicate is fan-out work per day. Fifty thousand followers posting forty times a day costs more than two hundred thousand posting weekly, and the product of the two is what you actually pay for.



# Capping the pulls at twenty does not meet the promise it was chosen to meet.

At thirty pulls and a one-percent slow call,  $1 - 0.99^{30}$  puts twenty-six percent of pages on a slow dependency. Cap at twenty and it is eighteen percent — still eighteen times a 99th-percentile budget. The cap sounded like the answer and fails its own arithmetic, which is exactly why you run the check on your own fixes.

## THE CACHE IS WHAT BOUNDS IT

A celebrity's recent-posts list is written once and read five hundred million times, so those pulls hit cache and sit nowhere near one percent slow.

## THE DEADLINE IS THE BACKSTOP

Fire every pull, render at eighty milliseconds with whatever arrived, and let the stragglers land on the next fetch.

## A CAP STILL COSTS SOMEONE

A reader following seven and a half thousand accounts can have hundreds above the line. For them twenty is not trimming the quietest — it is most of their pull path.



# The celebrity is a protagonist of the product and the antagonist of your design.

The product exists partly for her. She is the reason a hundred million people opened the app this morning, and the reason the write path cannot be one code path. Both things are true at once, and holding both is what designing feels like.

FOR THE PRODUCT

*She is the most valuable account on the platform, and she must post instantly.*

FOR THE DESIGN

She is a five-hundred-million-edge node that breaks every uniform assumption.

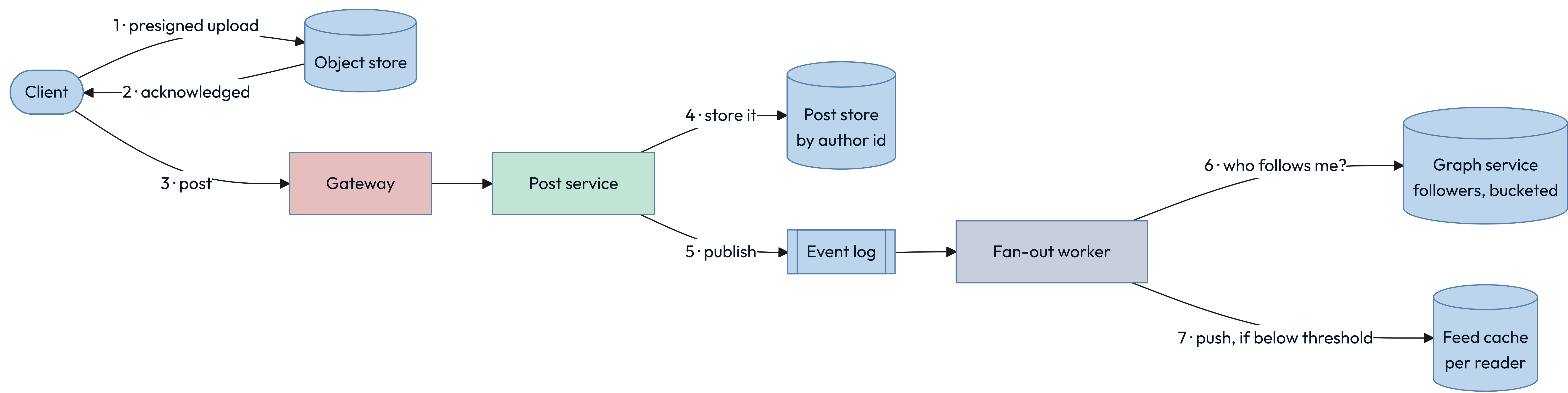
WHAT YOU DO ABOUT IT

You do not resolve the tension. You build a second path and name why it exists.



INSTAGRAM • THE WRITE PATH

The graph service is the box everyone forgets to draw.

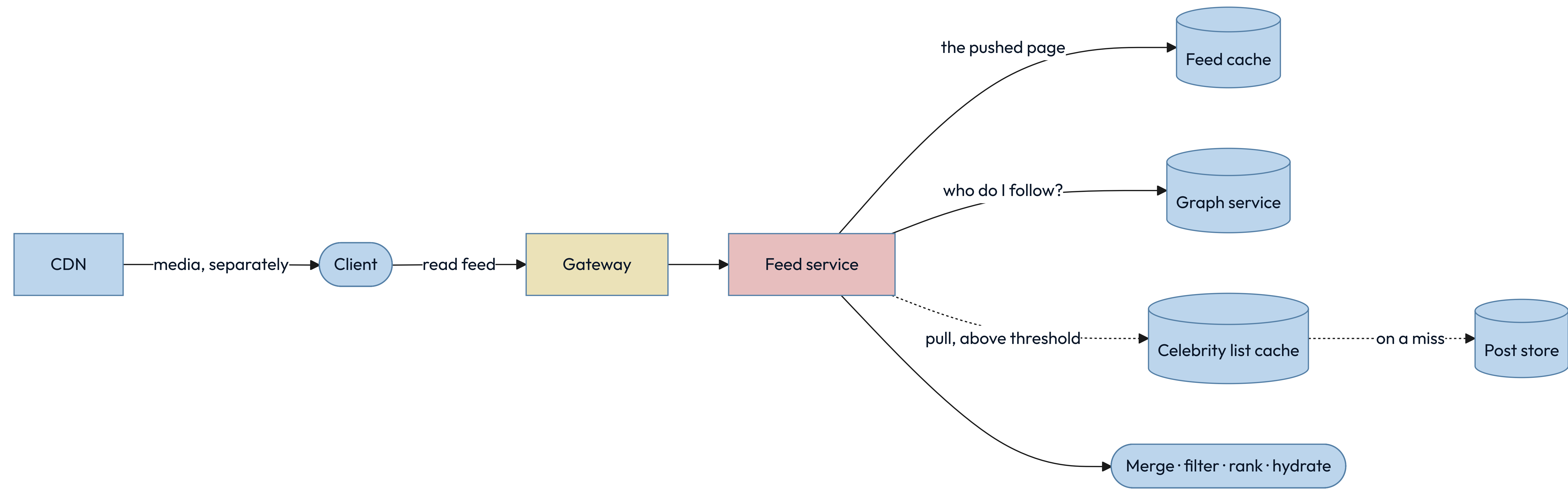


✦ Step 3 comes after step 2 on purpose. Publish before the bytes are acknowledged and the fan-out delivers a post whose media does not exist yet, to five hundred million people, and every one of them sees a broken tile. Transcoding may still be running — a variant that is not ready falls back to the original — but the original must be there before anyone is told about the post.



INSTAGRAM • THE READ PATH

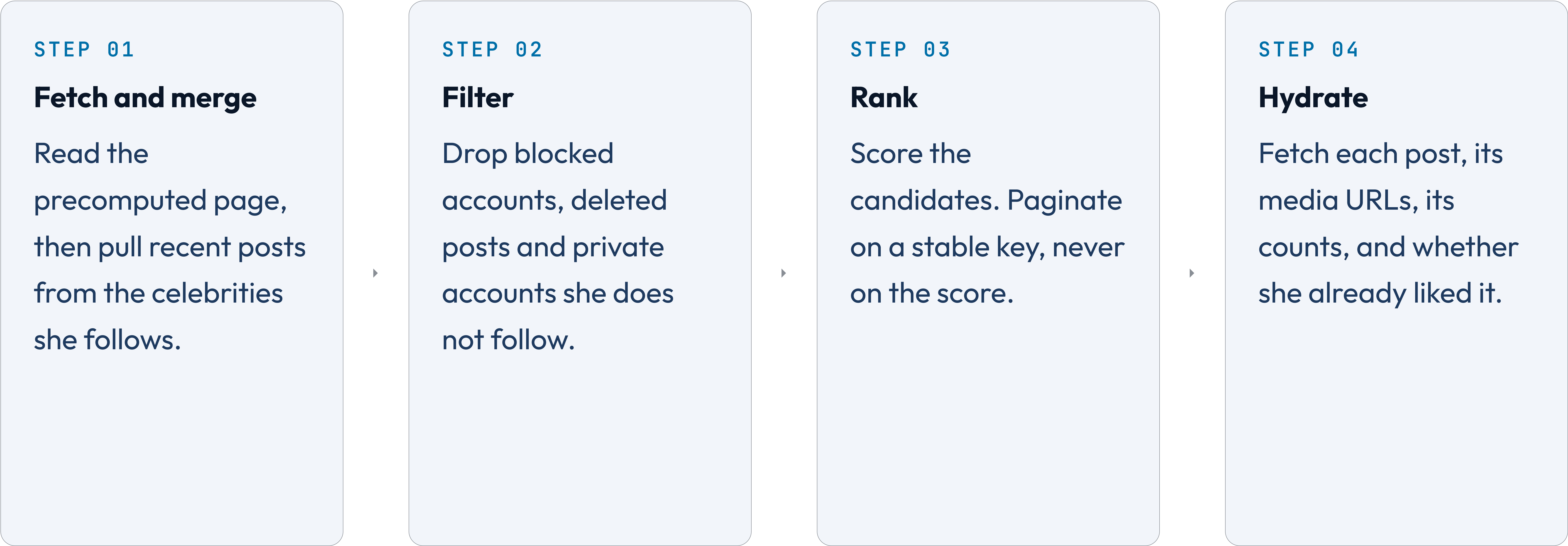
The read path is one lookup, plus whatever the celebrities added.



★ The pull path goes through a cache, not straight to the store. That box is the whole reason the hybrid is affordable: one celebrity's recent-posts list is written once and read five hundred million times, so it is the most cacheable object in the system. Draw it, or the design you hand someone sends half a billion reads a day at the post store.



# Hydration costs more than the other three stages together.





## THE NUMBER WE MISSED

# Every page you serve is eighty lookups behind it, so the read number is eighty million a second.

---

A twenty-item page needs four lookups an item — the post row, the media variants, the counts, and whether this reader liked it. Eighty a page, against the million page-fetches a second the peak estimate gave you, is eighty million.

Batch them all. Do not fold the count into the feed entry — that entry is written at post time, when the count is zero. Fold the like check the other way instead: one batched read across the twenty candidates.

Batching is not only throughput. Eighty parallel calls put  $1 - 0.99^{80}$ , fifty-five percent, of pages on a slow dependency — against a 99th-percentile promise.



INSTAGRAM • THE LIKELY BUG

# Your own post must appear instantly, or people think the upload failed.

The feed is eventually consistent, which is correct for everyone else's posts and completely wrong for your own. A person who posts and does not see it reads that as data loss, not as staleness, and posts again.

## WHERE IT COMES FROM

Read-your-writes, the consistency rung from Part four, applied to one reader's own posts.

## WRITE YOUR OWN FEED SYNCHRONOUSLY

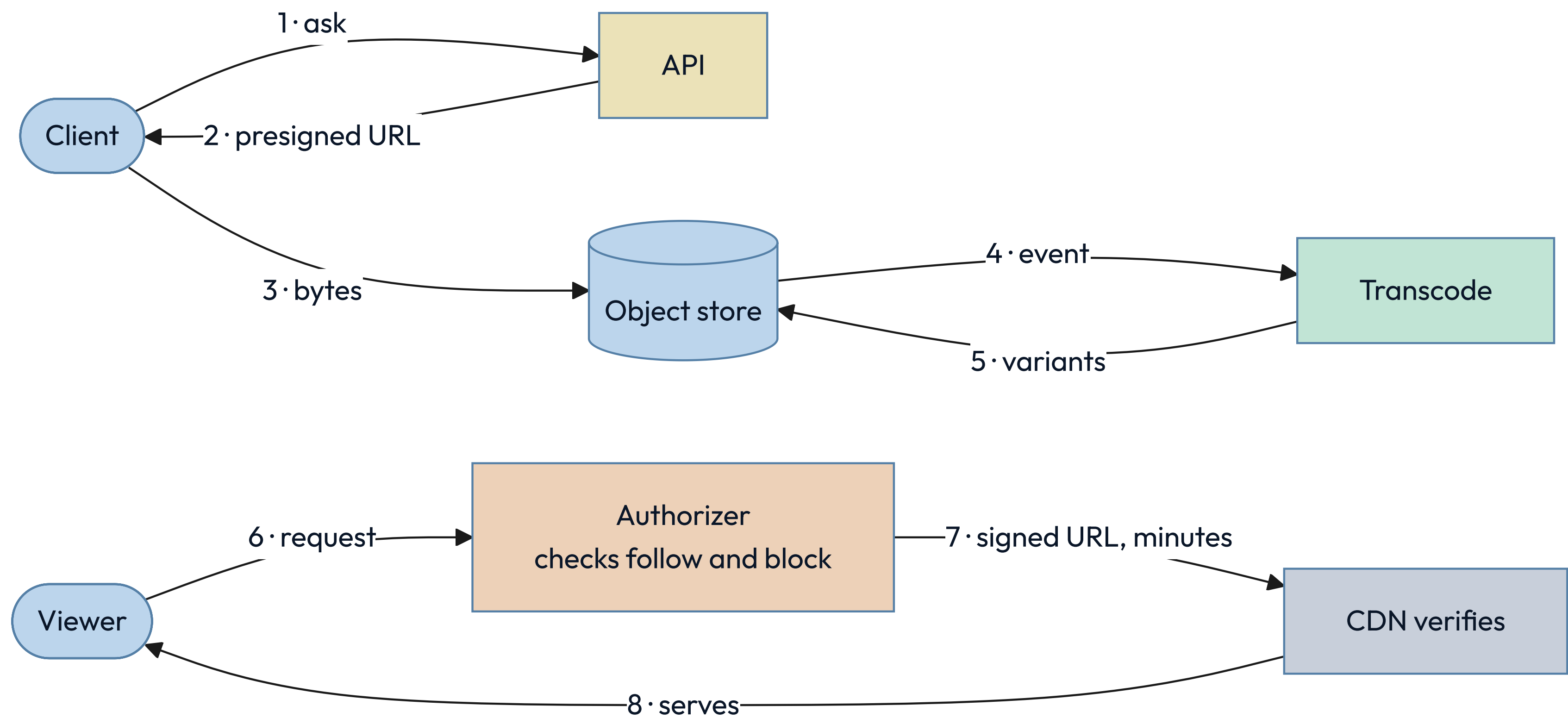
Inside the POST request, before it returns. One extra write, on one key.

## AND LET THE CLIENT HELP

It inserts the post it just created optimistically, and reconciles on the next fetch.



# Media never touches your API servers, and private media never touches an open URL.



✦ Step 2 is the one place an untrusted client writes straight into your storage. So the grant carries four limits: one content type, a size ceiling, a short expiry, and a rate per account. Without them, anyone can fill your storage at your expense, and step 4 hands bytes they chose to an image decoder. All input is untrusted, including input you asked for.



INSTAGRAM • THE PATH NOBODY DESIGNS

# A million likes on one post is the follower list again, on a different axis.

Every like is a write against the same post id — one key, one partition, one leader — arriving tens of thousands a second. That is the hot spot the data kit warned about, and the queue will sometimes deliver the same like twice.

## THE ROW IS THE TRUTH

Store the pair once, keyed by reader and post. The count is an aggregate you rebuild from it, which is what survives a redelivery or an unlike.

## SHARD THE HOT COUNTER ONLY

Increment one of a hundred sibling keys at random. One hot key becomes a hundred warm ones — and one read becomes a hundred, so cache the sum and shard only the keys you have measured as hot.

## COMMENTS ARE THE SECOND UNBOUNDED LIST

Bounded per reader, unbounded per post. Page them; never load them with the post.



# Every design gets drawn at rest. These four happen while it is running.

## A new follow

Their earlier posts were pushed before you existed. Backfill a few, or state the gap.

## An unfollow

Pushed entries do not remove themselves. Filter at read against the current edge.

## The same post twice

A crossed threshold leaves it in the pushed page and in the pull. Dedupe at the merge.

## A cold celebrity key

A million readers miss it at once. One refills it; the rest take the stale copy.

### KEY INSIGHT

A feed is a cache of a relationship. Change the relationship, leave the cache alone, and the reader is shown something untrue.



# Four failures here are certain, and every one already has its answer.

---

## Celebrity post

The fan-out queue floods. Answer: the pull path above the threshold.

## Feed cache eviction

Cold readers cost a rebuild. Answer: rebuild lazily from posts and edges.

## Redelivery

The queue delivers twice. Answer: the feed entry is keyed on reader and post.

## Zone loss

A whole zone disappears. Answer: media replicated, every derived store rebuildable.



# Six sentences hold, or the design is not finished.

- |   |                                                                                    |                                 |
|---|------------------------------------------------------------------------------------|---------------------------------|
| ✓ | A post is visible to every eligible follower who reaches it.                       | eventually                      |
| ✓ | Media is never lost once an upload is acknowledged.                                | durable                         |
| ✓ | A like counts exactly once per reader and post.                                    | idempotent                      |
| ✓ | A feed page never repeats or skips an item.                                        | stable cursor + dedupe at merge |
| ○ | A blocked account is filtered at read — but a signed media URL outlives the check. | keep the TTL short              |
| ✓ | Every derived store rebuilds from posts and edges.                                 | rebuildable                     |



# 06

---

PART SIX

**Every choice came out of a kit,  
including the one we refused.**



# Each store-and-run entry landed somewhere specific in the design.

| KIT ENTRY          | WHERE IT LANDED          | WHY THAT ONE                                               |
|--------------------|--------------------------|------------------------------------------------------------|
| Wide-column        | The two edge tables      | One key, then a sorted list of who is on the other end     |
| Object store       | Photo bytes and variants | Large, immutable, fetched by key, kept for years           |
| Key-value          | The feed cache           | The key is known exactly and nothing is asked of the value |
| Durable log        | Post events into fan-out | Producers outpace consumers, deliberately                  |
| Stateless services | Feed and post service    | Any instance serves any reader; state lives elsewhere      |
| Bounded queue      | Fan-out and transcode    | Unbounded, one celebrity means an outage                   |



# The network, scale, reliability and security kits landed here.

| KIT ENTRY                  | WHERE IT LANDED                        | WHY THAT ONE                                                       |
|----------------------------|----------------------------------------|--------------------------------------------------------------------|
| Reduce                     | One cached list per celebrity          | Written once, read five hundred million times                      |
| Spread                     | Posts by author, edges by bucket       | The bucket bounds the biggest list we still fan out                |
| Defer                      | Fan-out on write, below the threshold  | Moves the cost out of the reader's 200 milliseconds                |
| CDN with versioned URLs    | Every photo variant                    | Same bytes for many readers, and purging is eventual               |
| Bulkhead                   | Fan-out workers kept off the read path | A burst of ordinary fan-out must not starve the pool serving feeds |
| Object-level authorization | Every hydration                        | The endpoint check passes; the object check stops the breach       |



# The most useful entry here is the one we refused.

Most juniors asked to design Instagram reach for a graph database, because the words "social graph" are right there. The data kit already answered it, fifty slides before this design began.

## WHAT THE CARD SAID

*Walk away when you have relationships but only ever join two hops.*

## WHAT THE FEED ACTUALLY DOES

Walks one graph hop, me to the people I follow, then a keyed lookup that is not a traversal.

## WHY THE REFUSAL MATTERS MOST

A kit that only says yes is a catalog. One that says no is a tool.



## THE ARTIFACT

**This is the whole method, and it fits on one page.**

|               |                                                       |
|---------------|-------------------------------------------------------|
| Protagonist   | _____ wants to _____ so that _____                    |
| Antagonist    | But _____ – and it is this big: _____                 |
| Purpose       | _____                                                 |
| Boundary      | In: _____ Out: _____                                  |
| Environment   | _____                                                 |
| Constraints   | physical _____ economic _____ human _____ legal _____ |
| Invariants    | 1 _____ 2 _____ 3 _____                               |
| Bottleneck    | _____ measured at _____                               |
| Solution type | MVP • scaled • optimized • optimal • specialized      |



THE REVIEW

# Six questions to ask of any design, starting with your own.

- ☐ Who is the protagonist, and who are we not designing for? casting
- ☐ What is the antagonist, and what number describes it? evidence
- ☐ Which rung of the ladder is this, and does everyone agree? frame
- ☐ What breaks first at ten times the load? scale
- ☐ What happens when each dependency is slow rather than down? failure
- ☐ What can one stolen credential reach? security



## WHAT TO DO ON MONDAY

# The last of these four is the one that teaches you.

## STEP 01

**Pick the unglamorous system**

Not a famous one.  
The service you were debugging on Thursday, or the pipeline nobody wants to own.

## STEP 02

**Cast it before you draw it**

Protagonist and antagonist first, one sentence each. If you cannot name them, you do not understand it yet.

## STEP 03

**Fill the other seven fields**

Let them argue with you. A field you cannot answer is the design question you have been avoiding.

## STEP 04

**Bring the page to your one-on-one**

Ask the person across from you which field you got wrong. That conversation is the whole point of learning this.



HOW TO THINK ABOUT SYSTEMS

Name the person, name the  
force, and the design follows.

*Maya never designed anything on Tuesday, and she  
used every word in Part one before she went to bed.*





# Glossary

A – P

| TERM | DEFINITION                                                                                                                                             |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| ACID | A transaction happens completely or not at all, leaves the rules intact, does not interleave visibly with others, and survives a crash once committed. |
| API  | The contract one system offers another — the operations, their inputs, and what they promise in return.                                                |
| BASE | ACID's loose counterpart: stay answerable under failure, let replicas disagree for a while, and converge afterward.                                    |
| CAP  | Split by a network fault, a system must answer with possibly stale data or refuse to answer.                                                           |
| CDN  | Caches placed near readers so bytes travel a short distance instead of crossing an ocean.                                                              |
| CI   | The build and test run that gates every change before it can merge.                                                                                    |
| DNS  | The global lookup that turns a name a human types into an address a packet can be sent to.                                                             |
| MVP  | The smallest build that puts a real answer in front of a real user.                                                                                    |
| PR   | A proposed change, open for review before it merges.                                                                                                   |